# DayNest — informe de diseño de la aplicación

**Proyecto:** Renta de inmuebles por días.
**Eslogan:** Tu espacio, tus días.
**Versión del informe:** 10 de octubre de 2026.
**Integrantes:** por completar con los dos nombres del grupo.
**Asignatura, institución y docente:** por completar.
**Entrega indicada en el enunciado:** 21 de noviembre de 2026, hasta las 18:00.

# Contenido


1. Propósito y alcance
2. Requisitos y trazabilidad
3. Tecnologías y arquitectura
4. Modelo de datos y reglas
5. Diagramas de diseño
6. Fichas ampliadas de casos de uso
7. Verificación y demostración
8. Entregables y preparación final
9. Referencias
10. Datos bancarios y residencia
11. Extension financiera y estado de activacion
12. Comisiones e ingresos por inmueble

# 1. Propósito y alcance

DayNest es una aplicación web en Java conectada a PostgreSQL que permite publicar inmuebles para alquiler por días, buscar alojamientos, reservar, registrar pagos simulados y gestionar giros simulados al propietario. La interfaz distingue visitante, cliente, propietario y administrador. Una identidad puede reunir perfiles de cliente y propietario; el administrador se provisiona fuera del registro público.

Este informe documenta el código actual y reúne los diagramas de casos de uso, clases, interacción mediante secuencias y actividades exigidos por el enunciado. Añade el modelo de base de datos como soporte del diseño. Los PDF de clase se utilizan como guía de notación y fichas; no constituyen una rúbrica exhaustiva del profesor.

El alcance financiero es académico y local: aprobar un cobro, devolverlo o ejecutar un giro registra una simulación en la base de datos. No hay pasarela, banco ni conversión monetaria. La comisión es configurable (30 % inicial, editable por el administrador) y su porcentaje e importe se conservan en cada giro. Las fotografías son ilustrativas locales asignadas por identificador; no hay subida de fotos, reseñas, recuperación de contraseña implementadas; los datos y cuentas de los perfiles se editan desde Mi cuenta. La clasificación tipo/campo/mar sí es persistida, puede estar pendiente y se modifica por el propietario.

# 2. Requisitos y trazabilidad

Los identificadores RF se asignan en este informe para organizar la trazabilidad; no son números publicados por el profesor. RF08, RF10 y RF11 son ampliaciones implementadas que acompañan el alcance mínimo.

| ID | Comportamiento verificable | Casos de uso |
|---|---|---|
| RF01 | Publicar, mantener y retirar oferta del propietario | CU10–CU12, CU15, CU18 |
| RF02 | Registro, autenticación y cierre de sesión con perfiles | CU01–CU03 |
| RF03 | Búsqueda por ubicación, fechas, precio, capacidad y clasificación; detalle | CU04–CU05 |
| RF04 | Crear, consultar y cancelar reservas con validación de fechas/capacidad | CU06–CU07, CU09, CU17 |
| RF05 | Pago, saldo, historial de movimientos y devolución simulados | CU07–CU09, CU17 |
| RF06 | Liquidar automaticamente la estancia finalizada con pago completo, comision historica y cuenta del propietario pactado; conservar auditoria y reintentos. | CU15, CU19 |
| RF07 | Interfaz web para los distintos usuarios | CU01–CU22 |
| RF08 | Bloquear y liberar fechas del inmueble propio | CU13–CU14 |
| RF09 | Supervisión administrativa de rentas, reservas, pagos y giros | CU16–CU19 |
| RF10 | Consulta separada del historial previo en solo lectura | CU20 |
| RF11 | Comisiones acumuladas e ingresos por inmueble, moneda y arriendo, con historial paginado | CU21–CU22 |

Además del comportamiento funcional, se protege la salida JSP mediante JSTL, las operaciones de escritura mediante token CSRF, el acceso mediante sesión y rol, y la persistencia mediante consultas parametrizadas y transacciones. No se atribuye a estos controles una certificación formal de seguridad.

# 3. Tecnologías y arquitectura

| Área | Tecnología y responsabilidad |
|---|---|
| Lenguaje | Java 17 |
| HTTP | Jakarta Servlet, JSP y JSTL; servidor Jetty configurado por Maven |
| Presentación | HTML, CSS y JavaScript propios; fuentes, fotos y SVG locales |
| Negocio | Interfaces service e implementaciones; validación y coordinación transaccional |
| Persistencia | DAO JDBC y PostgreSQL; SQL parametrizado, restricciones y triggers |
| Construcción | Maven y wrapper mvnw; empaquetado WAR DayNest |
| Verificación | JUnit y comprobación HTTP; esquemas temporales para pruebas de datos |
| Documentación | PlantUML local con Smetana y Graphviz integrado para casos de uso; SVG y PDF |

El recorrido principal es navegador → servlet → service → DAO JDBC → PostgreSQL. Las JSP muestran resultados preparados por los servlets. Las interfaces permiten desacoplar contratos de negocio y persistencia. ServicioBase concentra operaciones comunes; UnidadTrabajoDaoJdbc entrega un contexto Daos con una misma conexión, confirma al terminar y revierte ante error.

La arquitectura tiene detalles concretos: CatalogoService consulta CiudadDao; BloqueoService y HistorialAnteriorService son clases de servicio propias, no implementaciones de interfaces inventadas. Algunas implementaciones instancian CiudadDaoJdbc o UsuarioDaoJdbc directamente. Los diagramas conservan esas dependencias donde se muestran, sin afirmar una inyección de dependencias uniforme inexistente.

# 4. Modelo de datos y reglas

El destino de sql/instalacion/schema.sql contiene diez tablas del profesor (propietarios, inmuebles, ciudades, paises, banco, clientes, asocia, relaciona, vincula y renta) y nueve extensiones tecnicas. La nomenclatura ya esta aplicada; la extension financiera esta aplicada en la base operativa. Cliente y propietario son entidades separadas con acceso tecnico compartido. Las vistas preservan contratos sin duplicar datos. El esquema completo esta en esquema_relacional.md.

Usuario es el acceso tecnico compartido; clientes y propietarios tienen entidades y cuentas bancarias separadas. Ocupacion pertenece exactamente a renta o bloqueo. El calendario usa [inicio, fin): la salida no ocupa noches. La exclusion GiST impide cruces de ocupaciones activas.

El catálogo precargado contempla 32 capitales colombianas y Cartagena, Murcia, España. Las etiquetas muestran ciudad, región y país; los identificadores distinguen homónimos. La búsqueda por texto normaliza tildes y mayúsculas. Los importes conservan COP o EUR según la reserva y se resumen por moneda.

| Regla | Diseño implementado |
|---|---|
| Importe | Total pactado = noches × tarifa histórica |
| Saldo | Total pactado − (cobros aprobados − devoluciones aprobadas) |
| Reserva | Entrada posterior a hoy, salida mayor, huéspedes positivos dentro de capacidad |
| Vencimiento | PENDIENTE_PAGO vence a los 30 minutos; proceso cada 60 segundos libera fechas |
| Confirmación | Pago completo; el botón liquida el saldo y simula aprobación |
| Cancelación | Solo pendiente/confirmada antes de entrada; libera ocupación y simula devolución si corresponde |
| Beneficiario | Propietario pactado histórico; un cambio posterior de dueño no lo reemplaza |
| Giro | Salida cumplida, reserva confirmada/finalizada, pago completo y ausencia de giro vigente; neto menos comisión histórica |
| Clasificación | Tipo y entornos opcionales; NULL es pendiente, no se deduce de ubicación/fotos |
| Historial | No se borran contratos y finanzas; public conserva el archivo del modelo anterior |

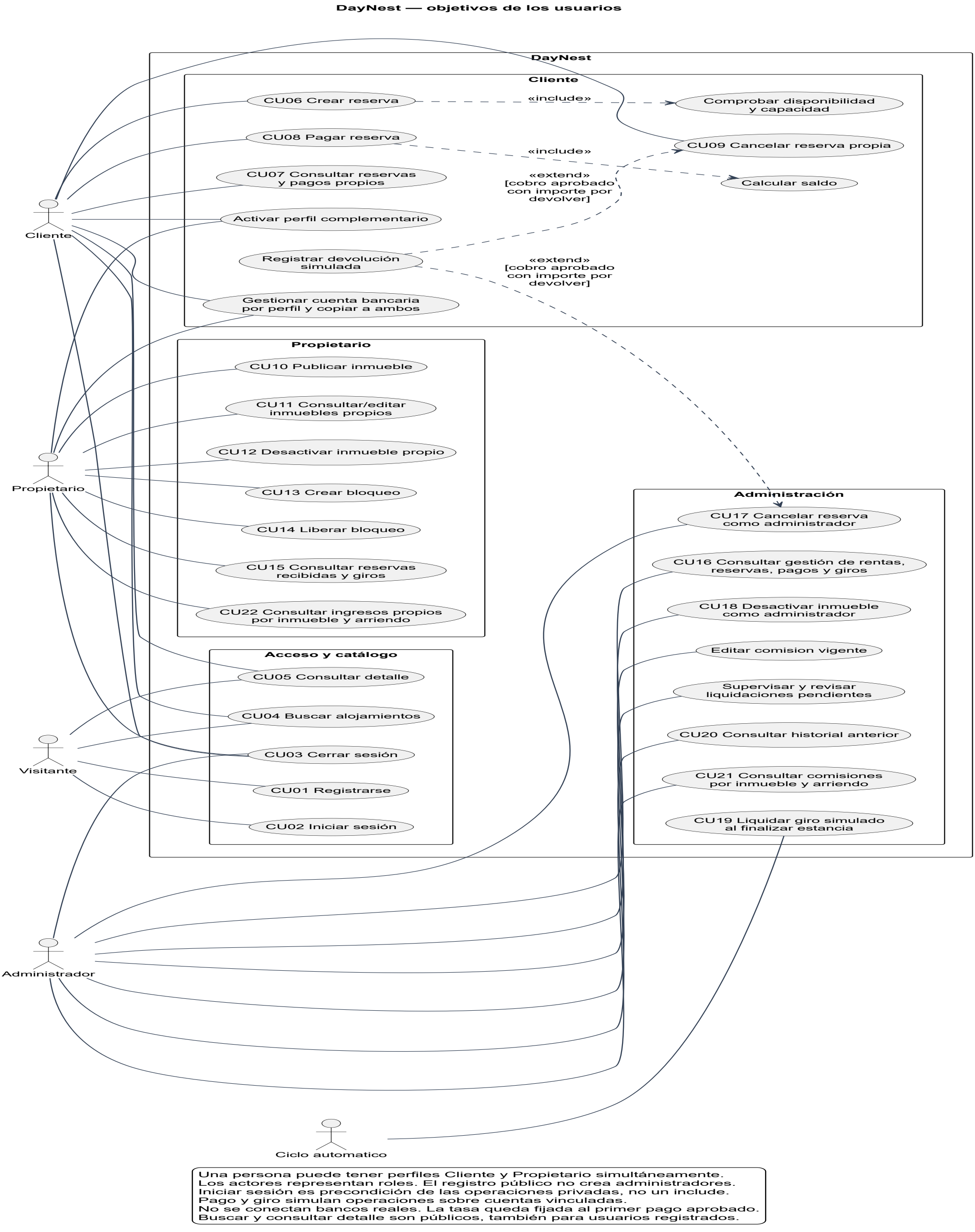
# 5. Diagramas de diseño

Las imágenes siguientes se generan desde las fuentes PlantUML del proyecto. Los métodos con (...) representan firmas abreviadas; no significa que exista un parámetro llamado puntos suspensivos. Se muestran operaciones relevantes, no todos los accesores ni todas las clases auxiliares. Las fichas CU01–CU22 describen los objetivos del usuario; las secuencias muestran su realización técnica.

## 5.1 Casos de uso

### Figura 1. Casos de uso de DayNest

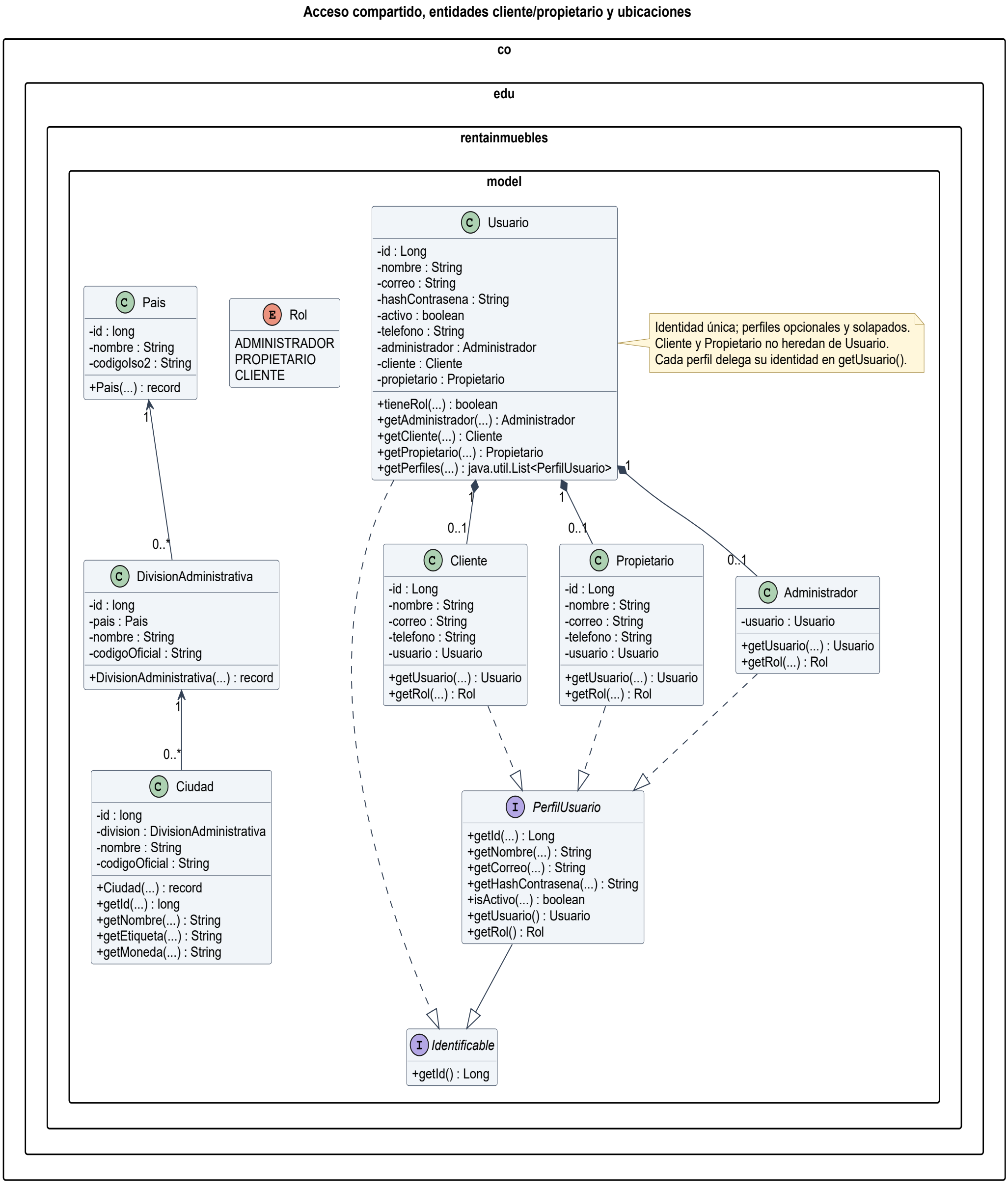
El límite DayNest separa sistema y actores. include representa un comportamiento obligatorio del caso base; extend representa la devolución condicionada a un cobro aprobado pendiente de devolver. Autenticarse se expresa como precondición de acciones privadas, evitando incluir un nuevo login en cada operación.



## 5.2 Clases

### Figura 2. Identidad, perfiles y ubicación

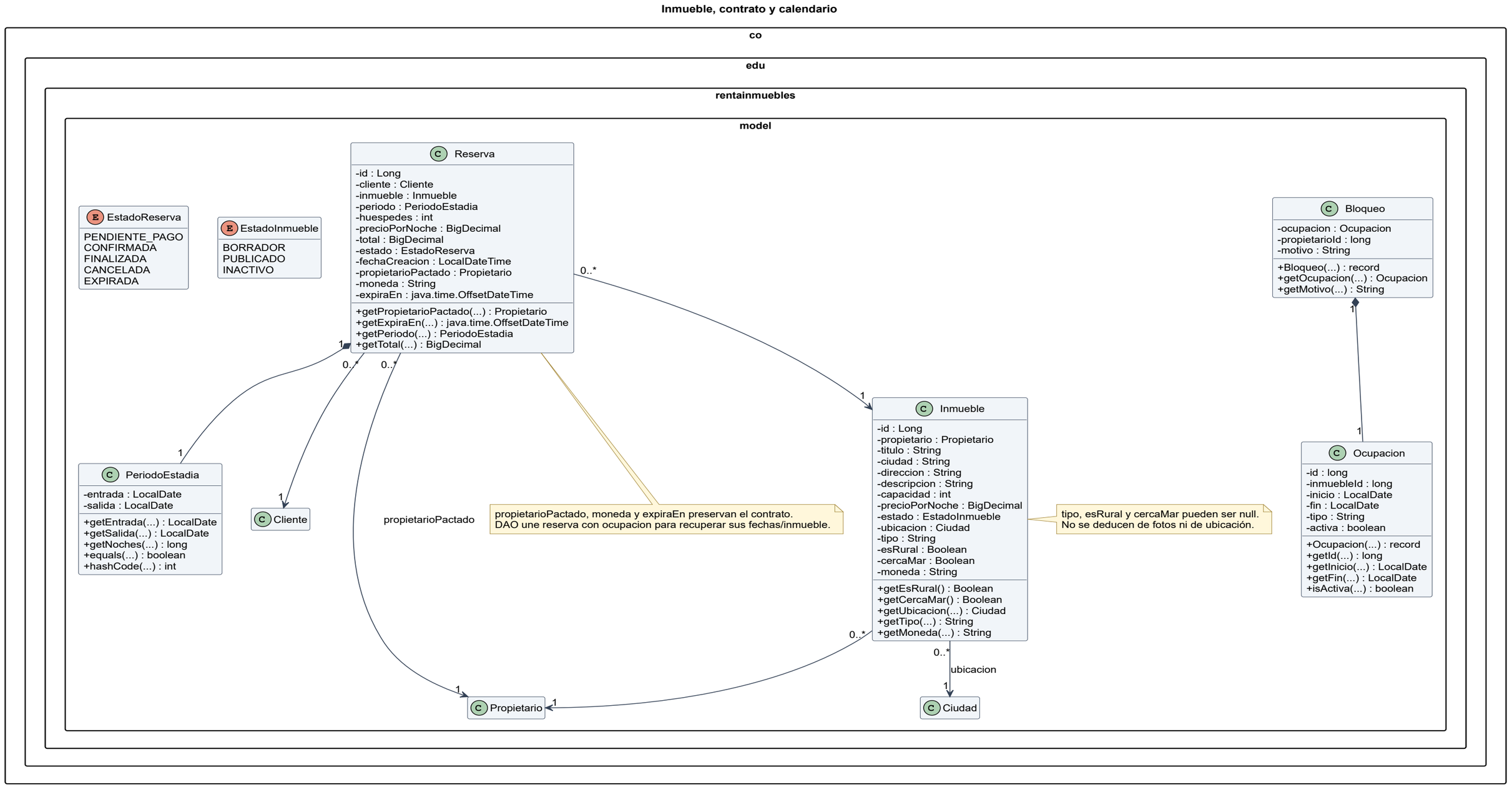
La identidad común reside en Usuario. Cliente, Propietario y Administrador implementan PerfilUsuario y delegan a la identidad; no son subclases de Usuario. Los registros geográficos representan país, división y ciudad, con relaciones de referencia y multiplicidad.




Firmas abreviadas (...); nombres y atributos comprobados contra Java.
Pago representa movimiento_pago. Datos geográficos: registros Java inmutables.

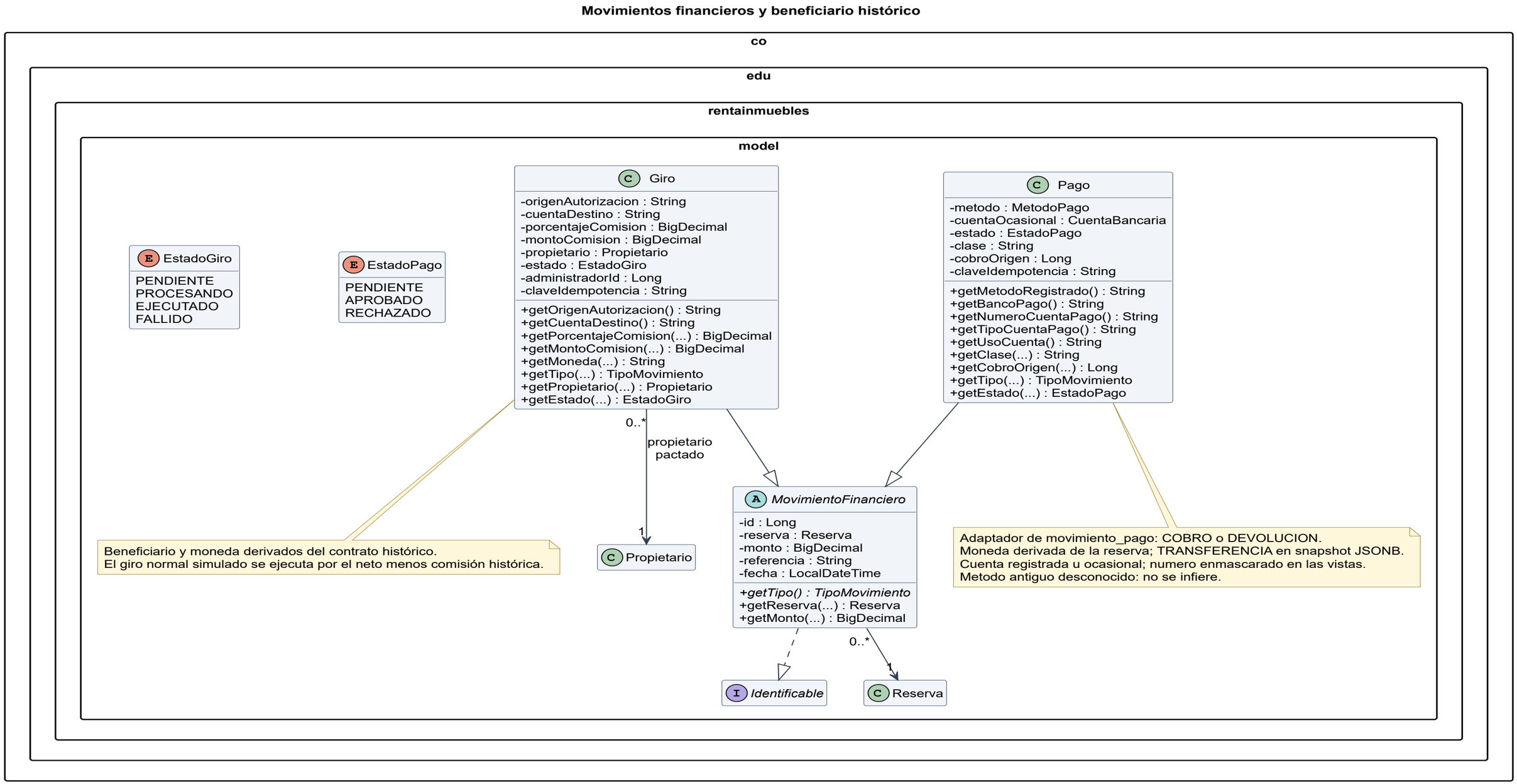
## Figura 3a. Clases de inmuebles, contrato y calendario

Inmueble incluye clasificación opcional y ubicación; Reserva conserva el contrato histórico y calcula total. MovimientoFinanciero es abstracta y se especializa en Pago y Giro. Pago adapta la tabla movimiento_pago; Giro obtiene beneficiario y moneda de la reserva. Ocupacion/Bloqueo representan el calendario técnico y el bloqueo del propietario.




Firmas abreviadas (...); nombres y atributos comprobados contra Java.

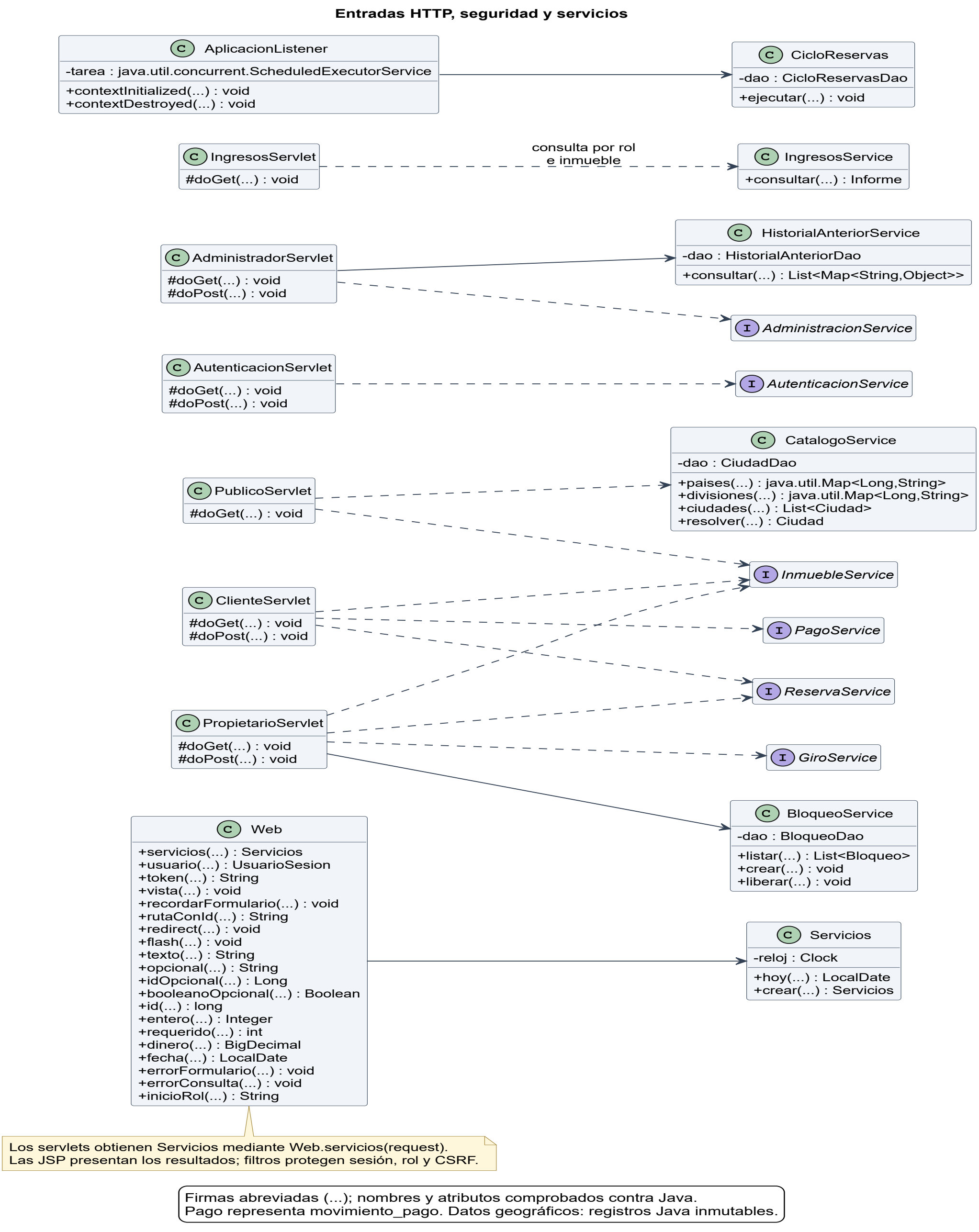
## Figura 3b. Clases de movimientos financieros




Se muestran operaciones principales; firmas abreviadas (...).

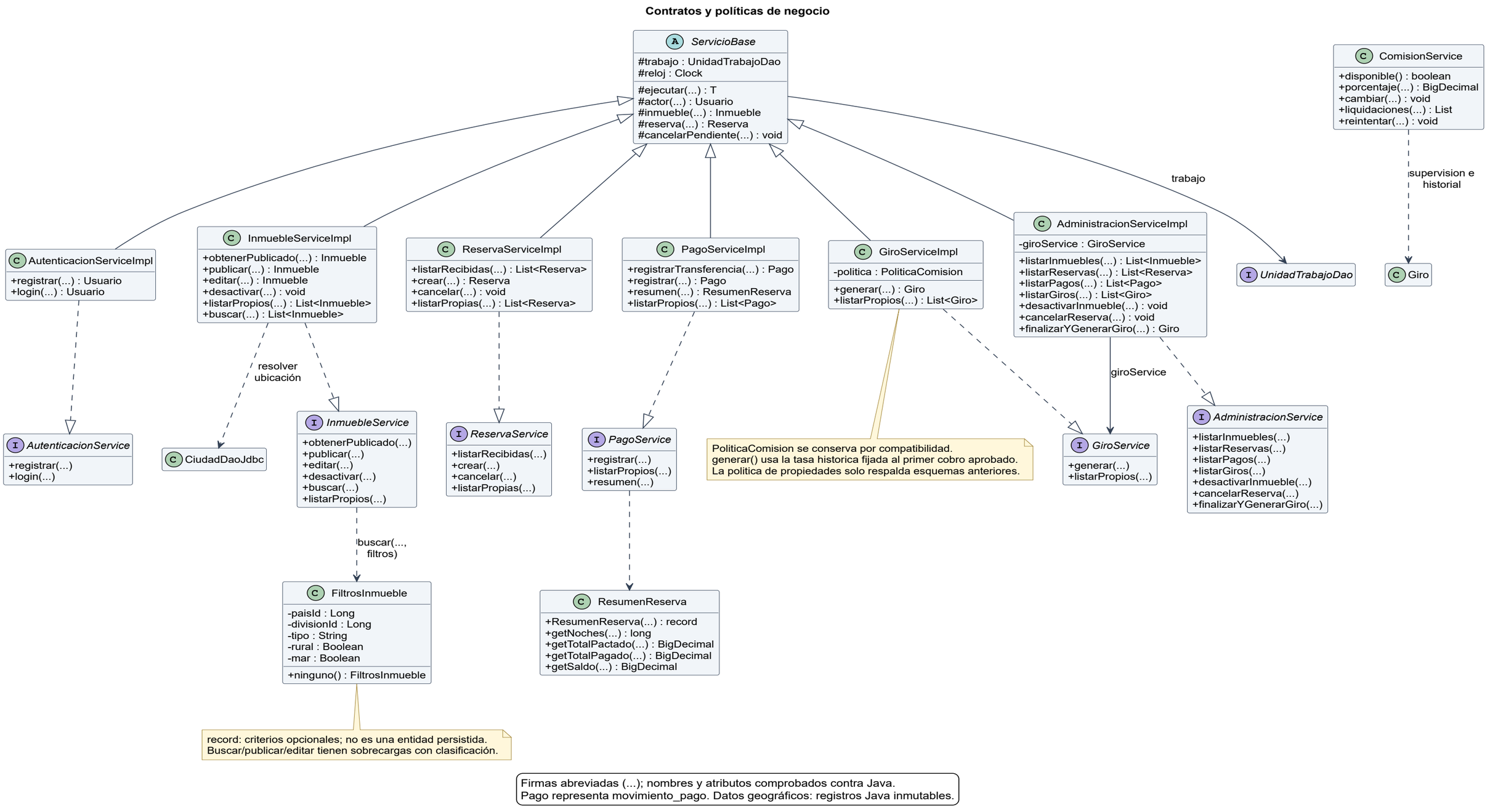
## Figura 4. Servlets y acceso a servicios

Las entradas HTTP se reparten por área pública, autenticación, cliente, propietario y administración. Web concentra utilidades HTTP y acceso a Servicios; AplicacionListener programa el ciclo periódico.



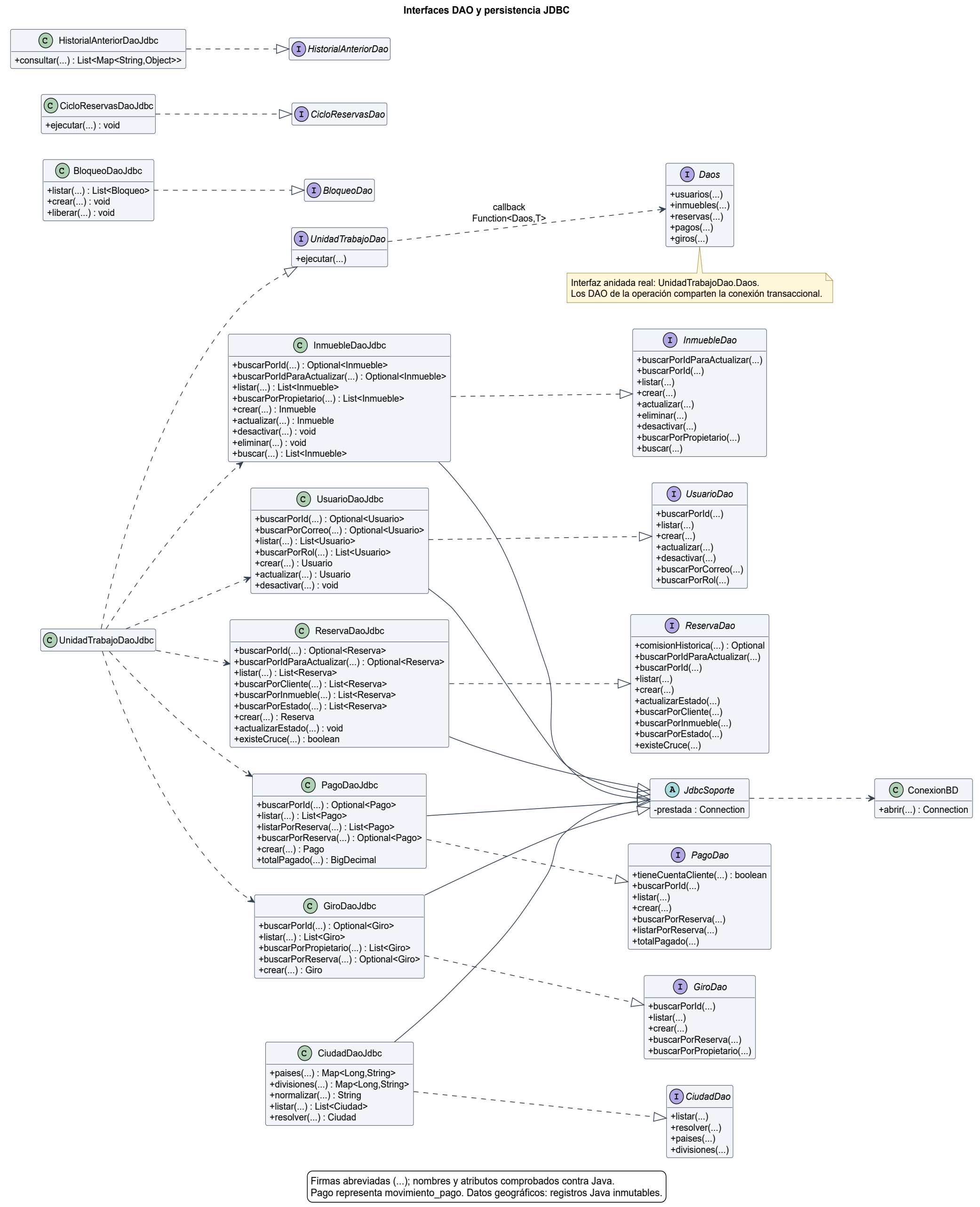
## Figura 5. Servicios, interfaces e implementaciones

Los contratos de negocio se realizan mediante sus implementaciones. FiltrosInmueble es un record de criterios, no una tabla nueva; ResumenReserva representa un resultado de cálculo. ServicioBase coordina validaciones comunes y la unidad de trabajo.



## Figura 6. DAO y persistencia JDBC

La persistencia separa interfaces de sus implementaciones JDBC. Daos es una interfaz anidada real de UnidadTrabajoDao; cada ejecución recibe DAO vinculados a la misma conexión. JdbcSoporte es abstracta.

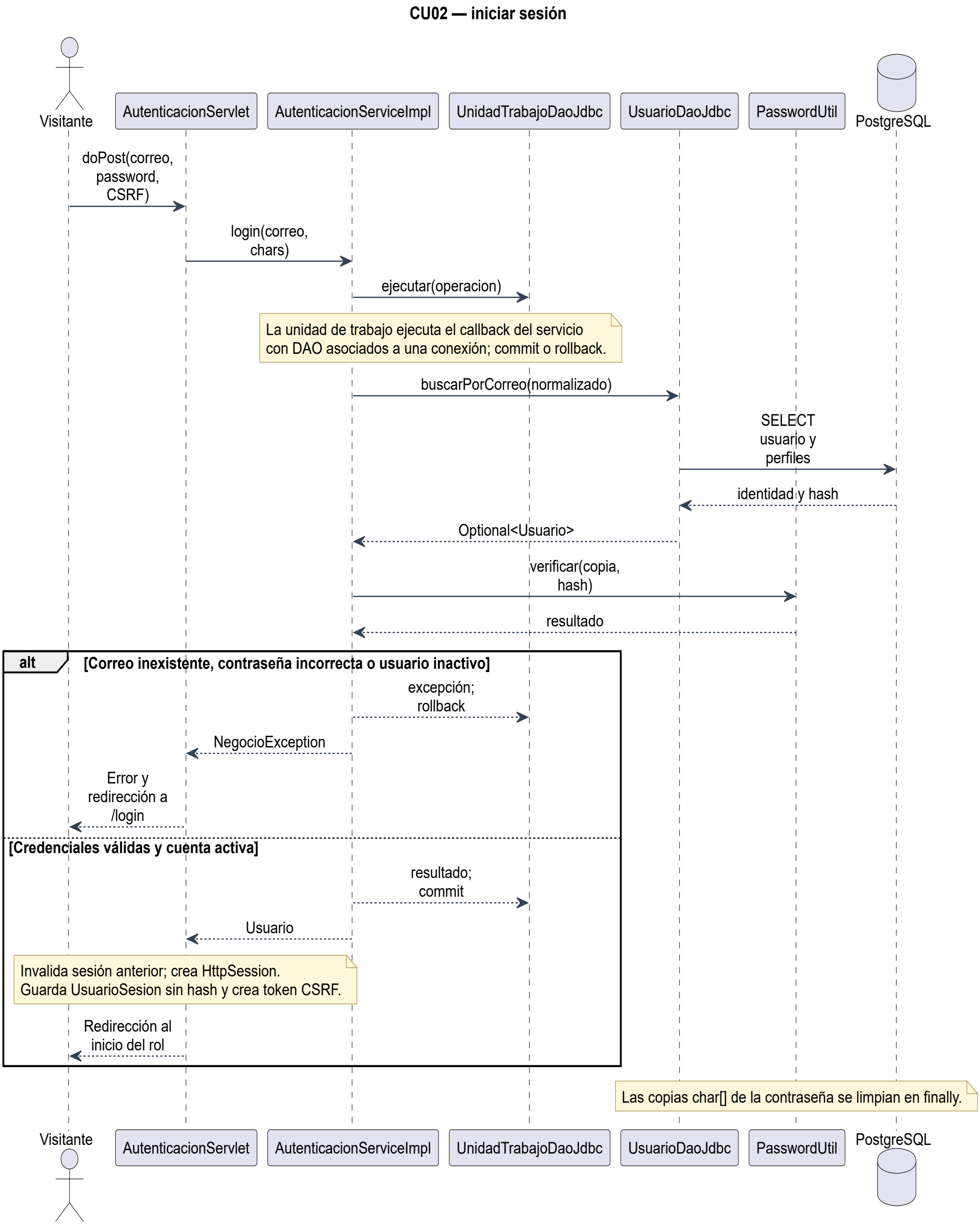


## 5.3 Interacción: diagramas de secuencia

### Figura 7. CU02: inicio de sesión

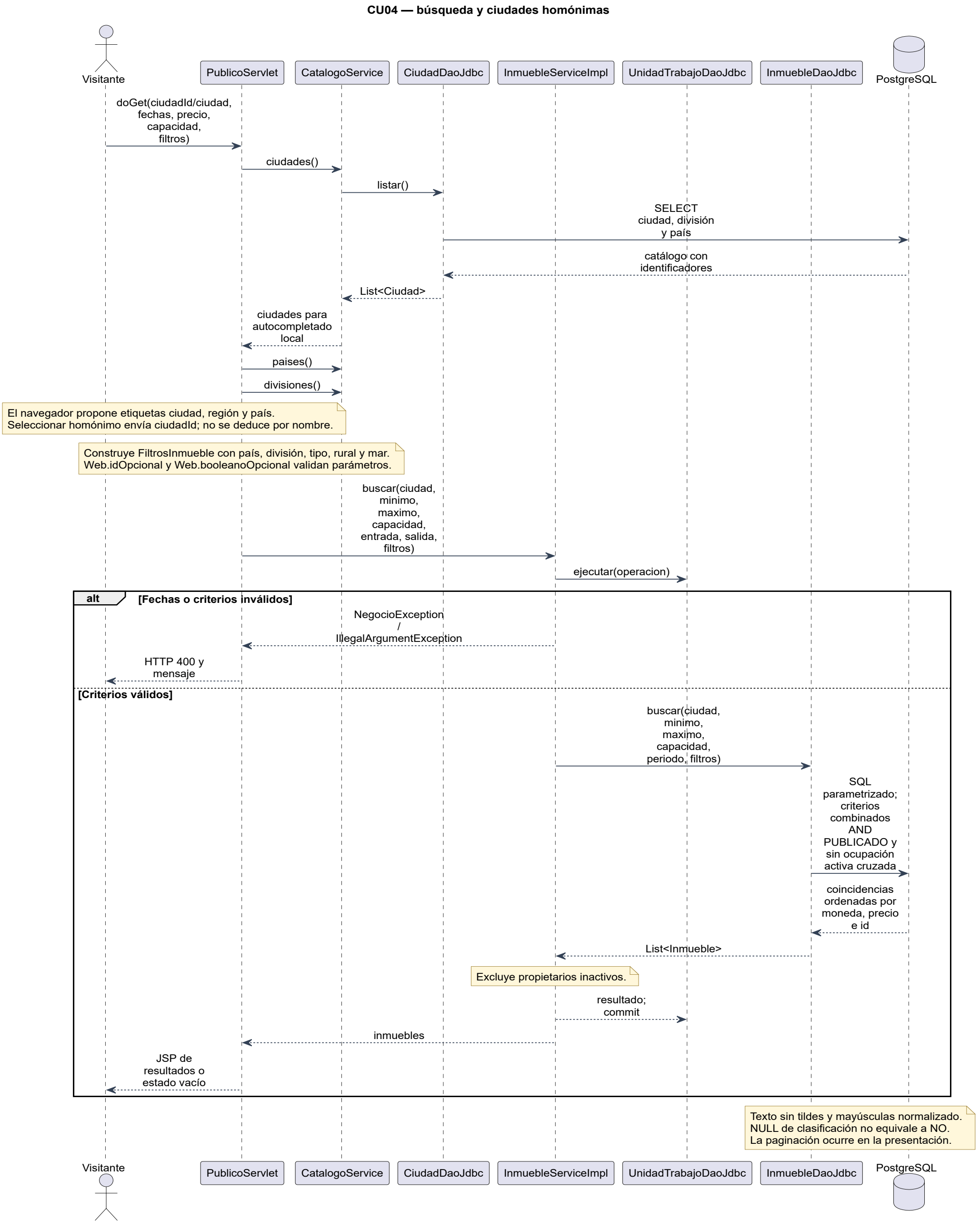
Las secuencias muestran orden de mensajes y alternativas de error para cinco recorridos centrales. El callback de la unidad de trabajo ejecuta el cuerpo del servicio; los mensajes SQL son descripciones, no métodos Java añadidos. commit/rollback son resultados de coordinación transaccional. Filtros HTTP de sesión, autorización y CSRF preceden a las operaciones privadas, aunque no se repiten como participantes en cada imagen.

La autenticación consulta identidad y perfiles, verifica contraseña y cuenta activa. Solo después establece una nueva sesión sin copiar el hash al objeto de sesión.



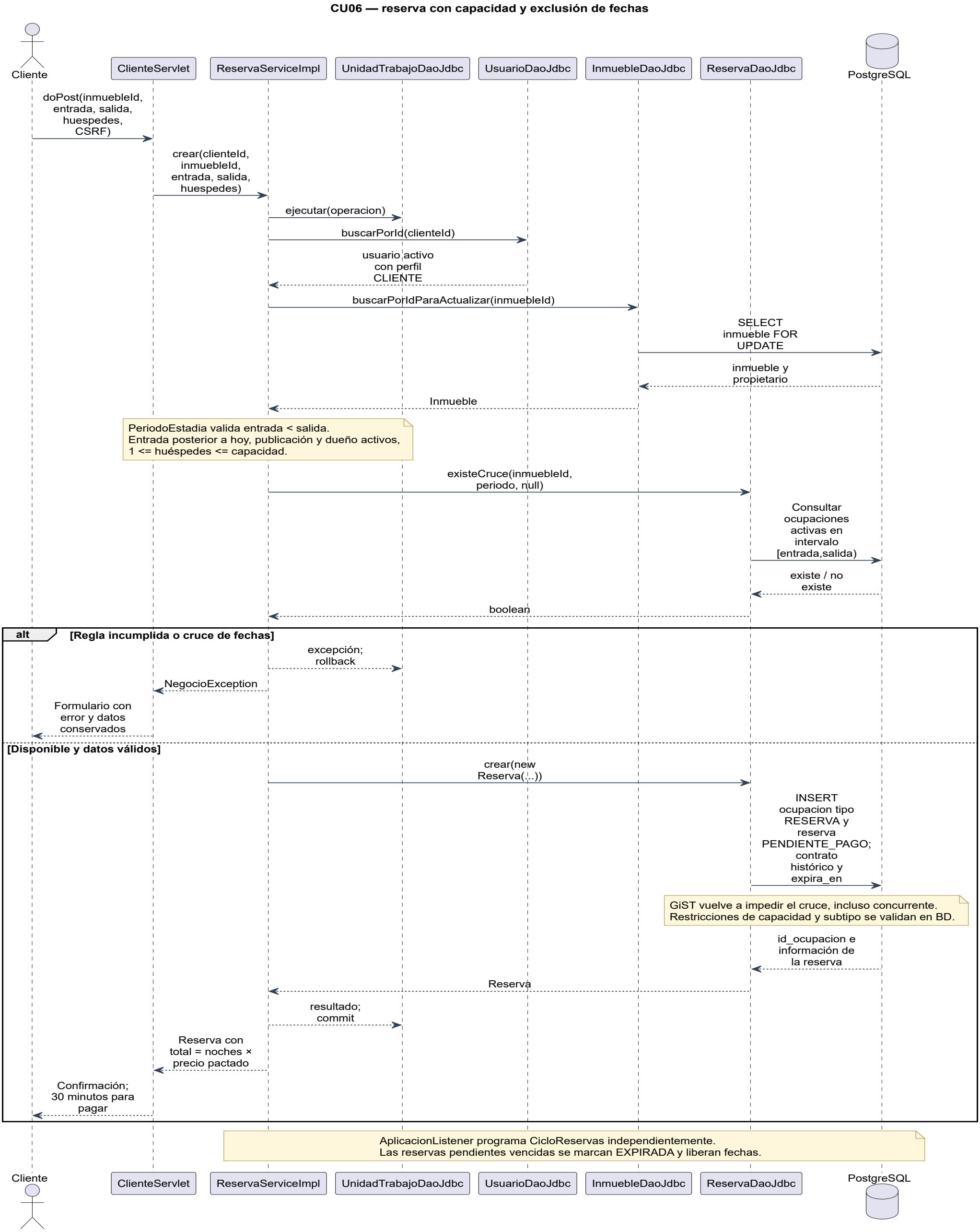
## Figura 8. CU04: búsqueda y catálogo geográfico

La búsqueda prepara catálogos y criterios combinados. El identificador seleccionado distingue ciudades homónimas y los entornos pendientes no se interpretan como valores negativos.



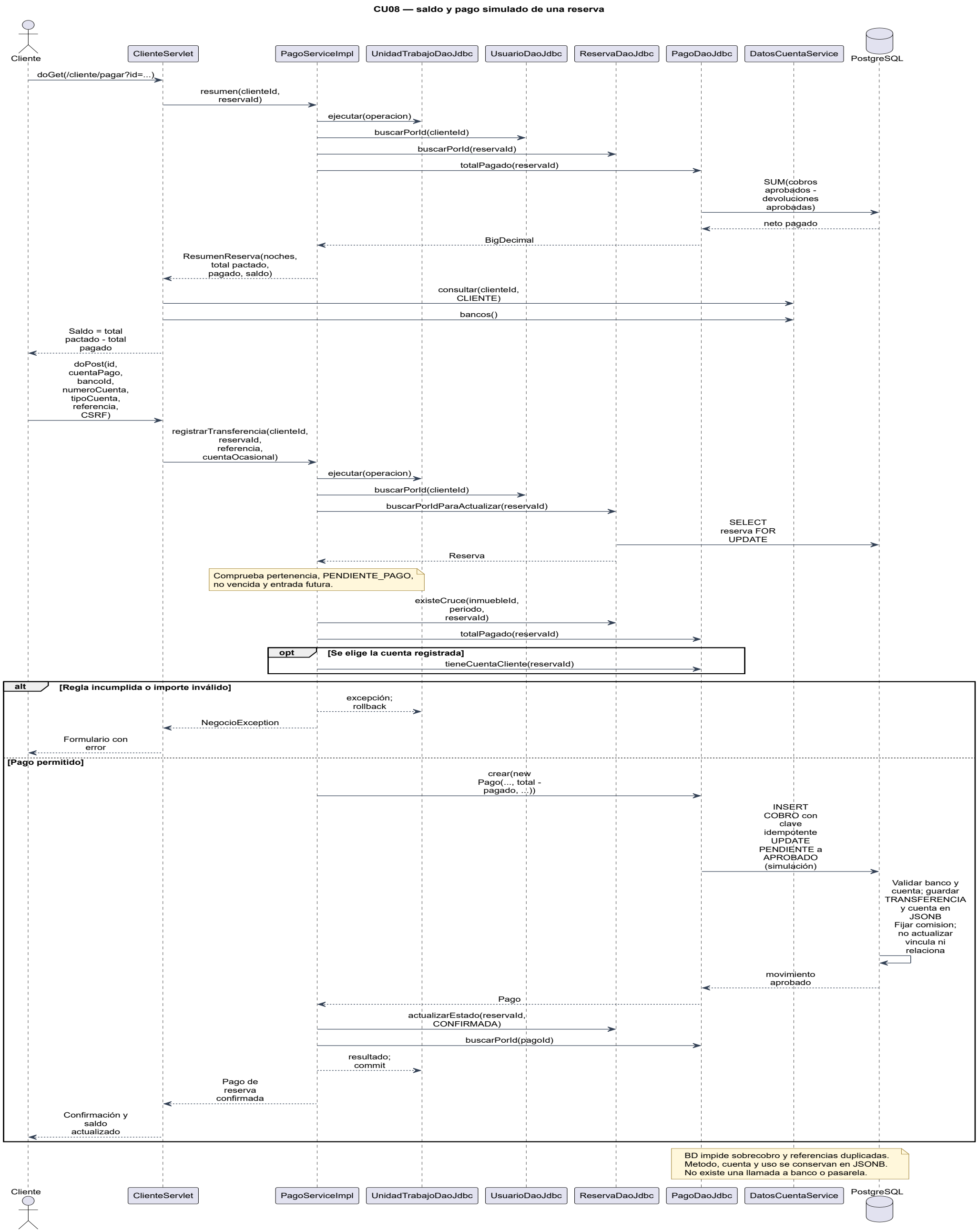
## Figura 9. CU06: creación de reserva

El bloqueo de inmueble, la comprobación de cruce y la restricción GiST complementan la validación de capacidad. Ocupación y contrato se guardan juntos.



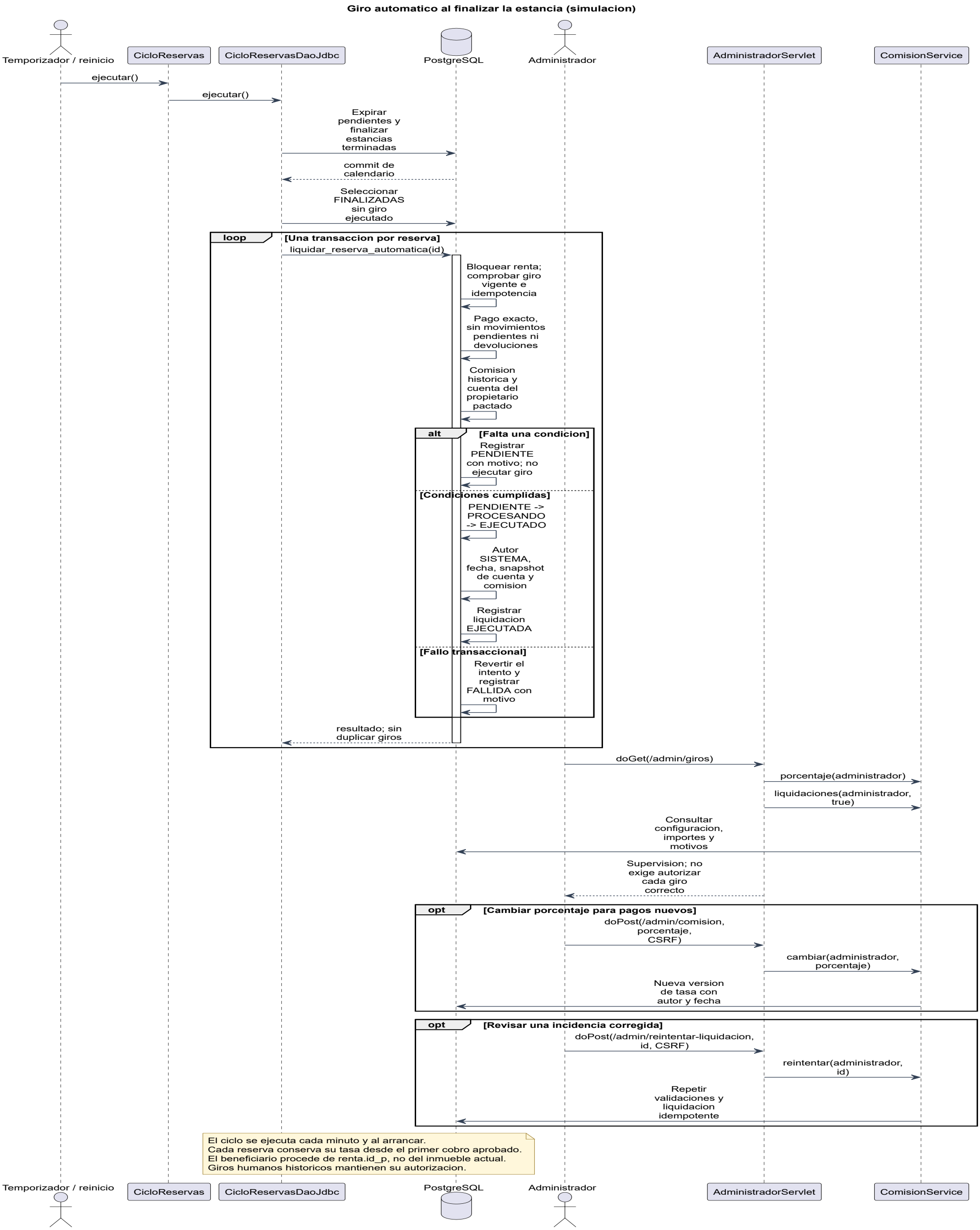
## Figura 10. CU08: cálculo de saldo y pago

Se separa la consulta de resumen del POST de pago. El saldo se recalcula en servidor con la reserva bloqueada; el movimiento se aprueba localmente y permite confirmar el contrato.



## Figura 11. CU19: generación de giro

El giro exige condiciones al momento de ejecutar, incluso si el botón ya aparecía disponible. La reserva pactada determina beneficiario y el neto aprobado determina importe.

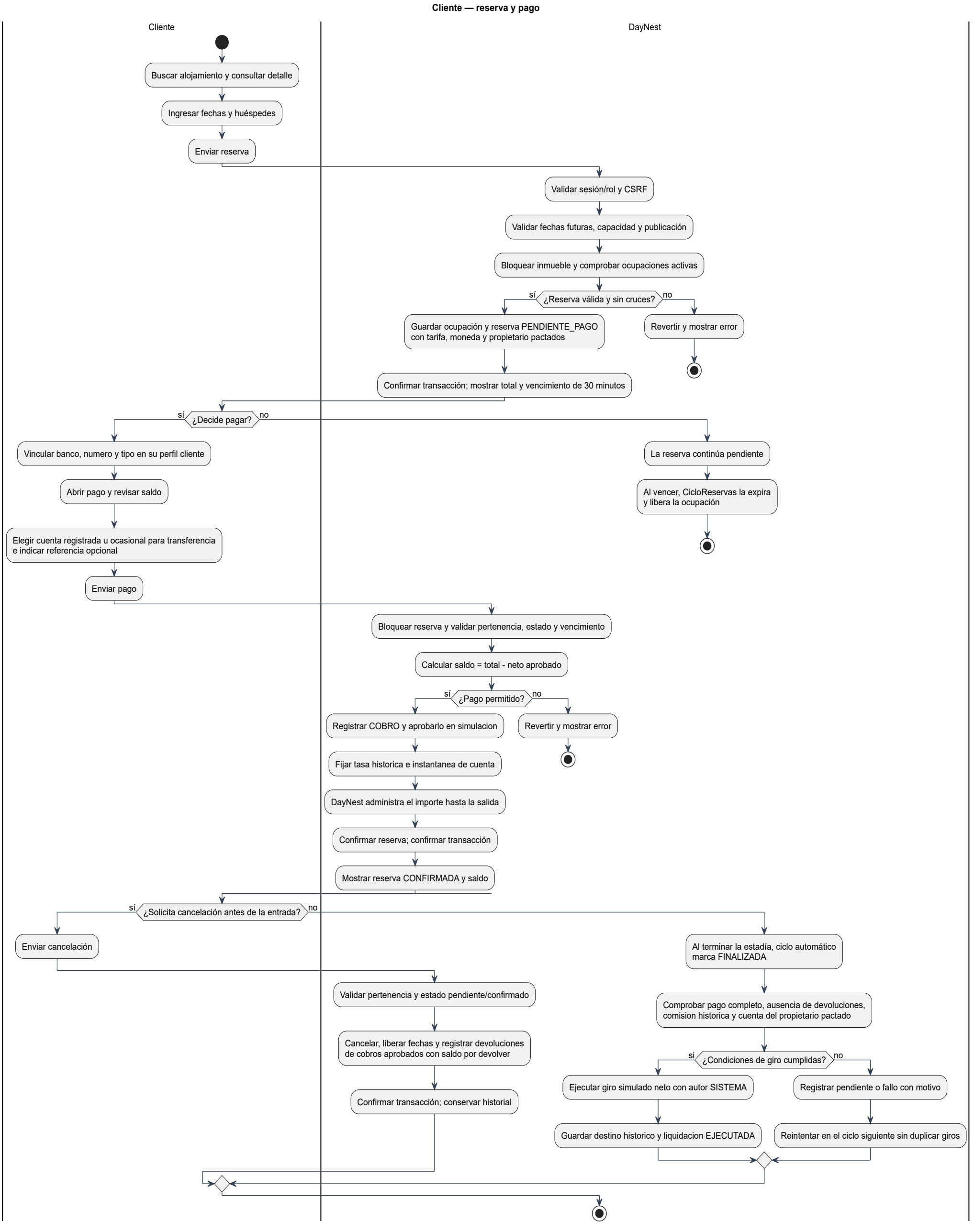


## 5.4 Actividades

### Figura 12. Reserva y pago del cliente

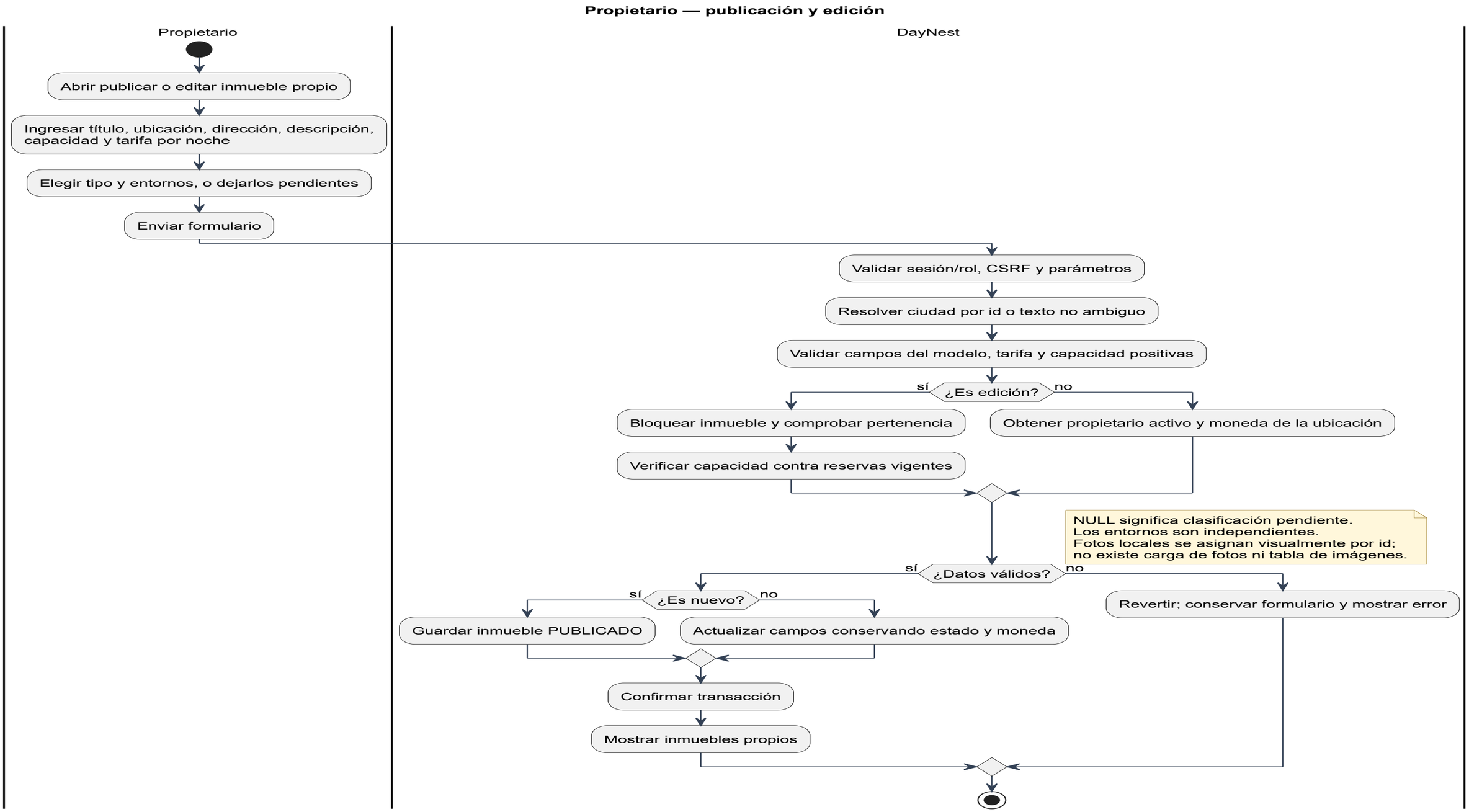
Las actividades distinguen responsabilidades del usuario y del sistema mediante carriles. Describen decisiones y resultados funcionales; no crean nuevos métodos o pantallas.

El recorrido abarca reserva, pago, vencimiento y cancelación previa a la entrada. El vencimiento ocurre posteriormente por el ciclo periódico, no inmediatamente por no pulsar pagar.



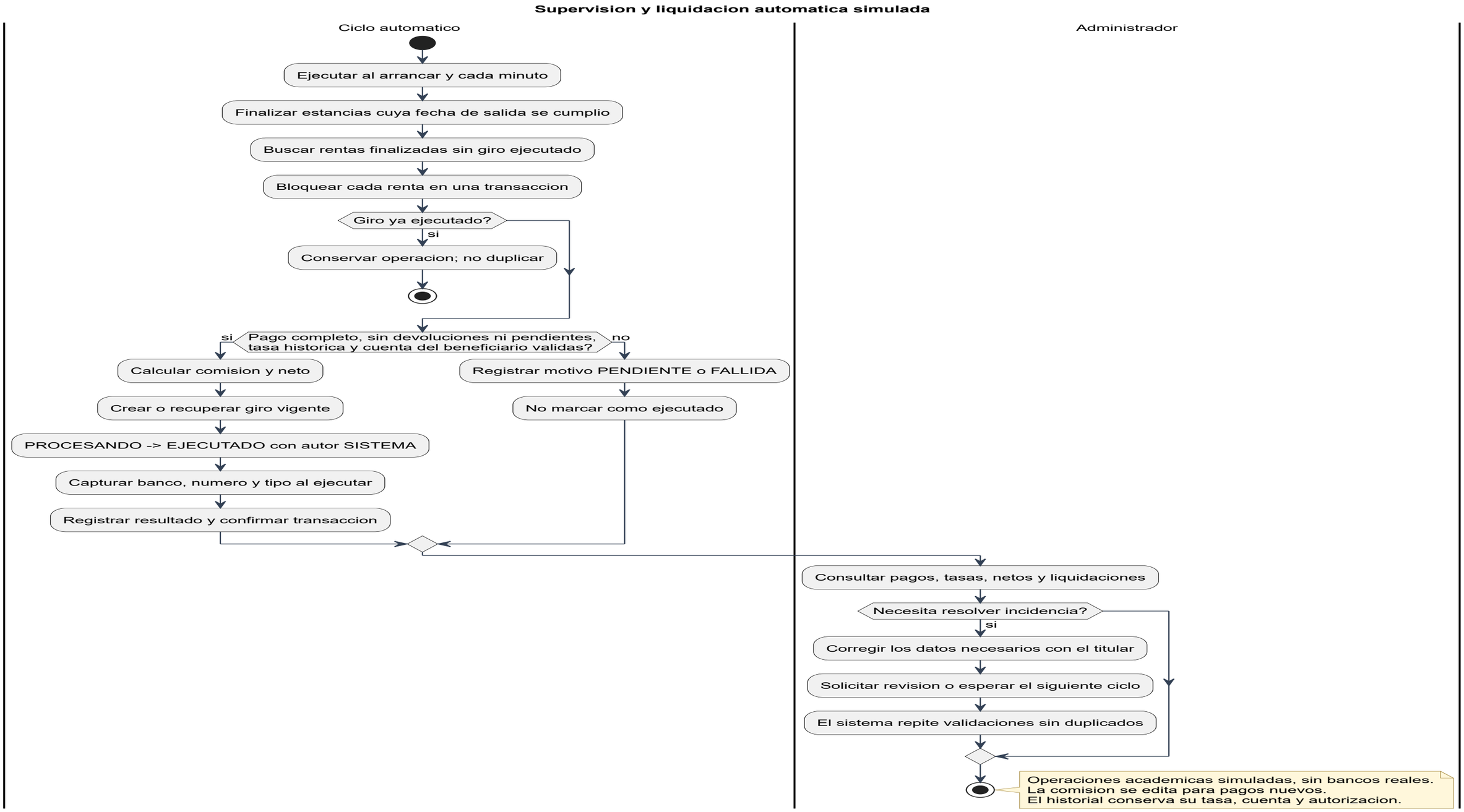
## Figura 13. Publicación y edición del propietario

Publicar crea un inmueble PUBLICADO; editar conserva el estado existente y verifica capacidad contra reservas vigentes. Los campos de clasificación pueden seguir pendientes.



## Figura 14. Gestión de giros del administrador

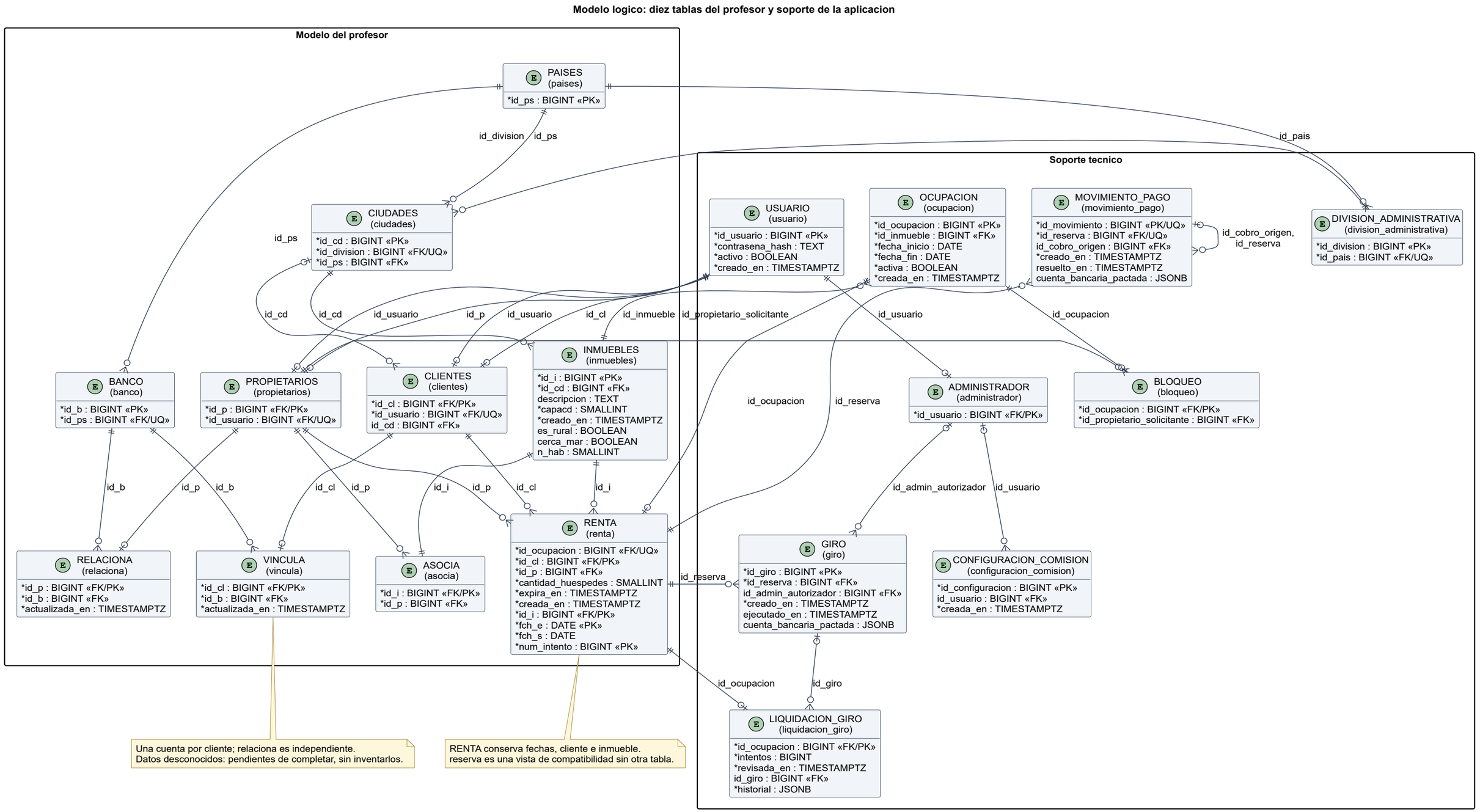
La actividad sigue el flujo de simulación implementado. El soporte de estados fallidos en la base no se presenta como una integración bancaria existente.



## 5.5 Modelo lógico del profesor y soporte técnico

### Figura 15. Modelo lógico PostgreSQL

Se presenta como apoyo a los diagramas de diseño solicitados, con todas las columnas, claves, cardinalidades y restricciones principales del sql/instalacion/schema.sql vigente.



# 6. Fichas ampliadas de casos de uso

Cada ficha sigue la estructura de la guía: nombre/identificador, actores, propósito, resumen, tipo, precondiciones, postcondiciones, referencias, curso normal Usuario/Sistema y alternativas. Todos los casos principales son de usuario y están descritos como casos reales sobre pantallas implementadas. include/extend del diagrama se detallan también en la sección 6.21.

Una precondición indica cuándo el caso puede terminar con éxito; el sistema vuelve a comprobarla al ejecutar. La garantía ante error de operaciones transaccionales es rollback. En acciones privadas, sesión, rol y CSRF se controlan mediante filtros además de las validaciones del servicio.

## 6.1 CU01 — Registrarse

**Actores e iniciador:** Visitante

**Propósito:** Crear una identidad con perfil cliente, propietario o ambos.

**Precondiciones:** No tener sesión iniciada; correo no registrado; elegir al menos un perfil público.

**Postcondiciones:** Éxito: usuario activo y perfiles seleccionados persistidos; se muestra el acceso. Fallo: no se guarda una identidad incompleta.

**Pantallas y salida:** /registro → /login

**Referencias:** RF02 · AutenticacionServlet.doPost; AutenticacionServiceImpl.registrar; UsuarioDaoJdbc.crear

**Resumen:** El actor inicia registrarse; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Abre el registro. | Muestra nombre completo, correo, teléfono opcional, contraseña y perfiles. |
| 2 | Completa y envía el formulario. | Valida datos y contraseña de 8 a 128 caracteres; normaliza correo y comprueba duplicados. |
| 3 | Espera el resultado. | Genera hash, guarda usuario y perfiles en una transacción y redirige al login. |

**Alternativas y errores:** 2a. Correo duplicado, contraseña inválida o perfil no permitido: mostrar error y conservar campos no sensibles. No se habilita registro de administradores. No se conserva la contraseña en el formulario.

**Relaciones:** Sin include/extend adicional.

## 6.2 CU02 — Iniciar sesión

**Actores e iniciador:** Visitante

**Propósito:** Autenticarse y acceder a las funciones del rol.

**Precondiciones:** Cuenta registrada; credenciales disponibles; cuenta activa.

**Postcondiciones:** Éxito: nueva sesión con UsuarioSesion y token CSRF. Fallo: no se establece sesión autenticada.

**Pantallas y salida:** /login → inicio del rol

**Referencias:** RF02 · AutenticacionServlet.doPost; AutenticacionServiceImpl.login; PasswordUtil.verificar

**Resumen:** El actor inicia iniciar sesión; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Abre el login y escribe correo y contraseña. | Muestra formulario con token CSRF. |
| 2 | Envía credenciales. | Normaliza correo, consulta identidad/perfiles y verifica el hash. |
| 3 | Accede a la aplicación. | Comprueba cuenta activa, invalida sesión anterior, crea sesión y redirige al inicio del rol. |

**Alternativas y errores:** 2a. Correo inexistente o contraseña incorrecta: mensaje de credenciales inválidas. 3a. Cuenta inactiva: acceso denegado. Las copias de contraseña se limpian en finally.

**Relaciones:** Sin include/extend adicional.

## 6.3 CU03 — Cerrar sesión

**Actores e iniciador:** Cliente, Propietario o Administrador

**Propósito:** Terminar la sesión del navegador actual.

**Precondiciones:** Sesión iniciada; envío mediante POST y token CSRF válido.

**Postcondiciones:** La sesión anterior queda invalidada y se vuelve a login.

**Pantallas y salida:** Formulario del encabezado → /salir → /login

**Referencias:** RF02 · AutenticacionServlet.doPost

**Resumen:** El actor inicia cerrar sesión; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Pulsa cerrar sesión. | Recibe POST /salir y valida CSRF mediante filtro. |
| 2 | Espera la salida. | Invalida la sesión, registra aviso y redirige a login. |

**Alternativas y errores:** 1a. GET /salir: HTTP 405. Token ausente o incorrecto: filtro rechaza la petición. El cierre no cancela reservas ni borra datos.

**Relaciones:** Sin include/extend adicional.

## 6.4 CU04 — Buscar alojamientos

**Actores e iniciador:** Visitante; también usuarios registrados

**Propósito:** Encontrar alojamientos publicados según criterios combinados.

**Precondiciones:** Catálogo disponible; no requiere autenticación.

**Postcondiciones:** Se muestran coincidencias o estado vacío; no se crea reserva ni se modifica inventario.

**Pantallas y salida:** /inicio y /buscar

**Referencias:** RF03 · PublicoServlet.doGet; CatalogoService.ciudades/paises/divisiones; InmuebleServiceImpl.buscar; InmuebleDaoJdbc.buscar

**Resumen:** El actor inicia buscar alojamientos; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Escribe ciudad o selecciona una sugerencia con región y país. | Carga catálogo local desde BD; al seleccionar conserva ciudadId para distinguir homónimos. |
| 2 | Añade país, región, fechas, rango de precio, capacidad, tipo o entornos y busca. | Valida parámetros y ambas fechas; construye FiltrosInmueble y combina los criterios con AND. |
| 3 | Revisa resultados y cambia filtros o página. | Consulta SQL parametrizado, excluye ocupaciones activas cruzadas y propietarios inactivos; presenta tarjetas y filtros activos. |

**Alternativas y errores:** 2a. Una fecha sola, entrada no futura o parámetro inválido: HTTP 400 con mensaje. 3a. Sin coincidencias: estado vacío. Texto sin tildes es normalizado; buscar Cartagena sin seleccionar id puede incluir ambos países. NULL de entorno es pendiente y no equivale a no. Precios se presentan en su moneda, sin conversión.

**Relaciones:** Sin include/extend adicional.

## 6.5 CU05 — Consultar detalle

**Actores e iniciador:** Visitante; también usuarios registrados

**Propósito:** Revisar un alojamiento disponible y estimar el precio de la estancia.

**Precondiciones:** Identificador de inmueble existente; inmueble PUBLICADO y propietario activo.

**Postcondiciones:** Se presentan datos reales y estimación; no se bloquean fechas hasta crear reserva.

**Pantallas y salida:** /inmueble?id=...

**Referencias:** RF03, RF07 · PublicoServlet.doGet; InmuebleServiceImpl.obtenerPublicado

**Resumen:** El actor inicia consultar detalle; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Selecciona una tarjeta. | Recupera inmueble publicado y sus datos. |
| 2 | Consulta ubicación, descripción, tarifa y capacidad; introduce fechas. | Muestra fotografías locales ilustrativas y estima noches × tarifa. |
| 3 | Elige reservar. | Enlaza al formulario privado de cliente; la reserva vuelve a validar condiciones en servidor. |

**Alternativas y errores:** 1a. Inmueble inexistente, inactivo o propietario inactivo: HTTP 404. Las fotos no acreditan servicios ni corresponden a una galería persistida. La estimación visual no sustituye el precio pactado en servidor.

**Relaciones:** Sin include/extend adicional.

## 6.6 CU06 — Crear reserva

**Actores e iniciador:** Cliente

**Propósito:** Contratar un inmueble por un intervalo de días y número de huéspedes.

**Precondiciones:** Cliente activo autenticado; inmueble publicado y dueño activo; entrada posterior a hoy; salida mayor que entrada; huéspedes positivos dentro de capacidad; intervalo libre.

**Postcondiciones:** Éxito: ocupación RESERVA y reserva PENDIENTE_PAGO con contrato histórico y vencimiento de 30 minutos. Fallo: rollback, sin ocupación parcial.

**Pantallas y salida:** /cliente/reservar?id=... → /cliente/reservas

**Referencias:** RF04 · ClienteServlet.doPost; ReservaServiceImpl.crear; ReservaDaoJdbc.existeCruce/crear

**Resumen:** El actor inicia crear reserva; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Abre reservar desde un inmueble. | Muestra formulario de fechas y huéspedes. |
| 2 | Confirma el formulario. | Verifica identidad/rol, crea PeriodoEstadia y bloquea el inmueble; comprueba publicación, dueño, fechas y capacidad. |
| 3 | Espera la confirmación. | Comprueba ocupaciones activas y guarda ocupación más reserva atómicamente; BD revalida exclusión y restricciones. |
| 4 | Consulta la reserva creada. | Presenta identificador, total = noches × tarifa pactada, moneda y plazo de pago. |

**Alternativas y errores:** 2a. Datos inválidos: error con formulario conservado. 3a. Cruce con reserva pendiente, confirmada o bloqueo activo: rechazar. Si otro cliente ocupa el intervalo concurrentemente, GiST impide confirmar ambas operaciones. Intervalos adyacentes son compatibles. Vencida sin pagar, el proceso periódico marca EXPIRADA y libera fechas.

**Relaciones:** Incluye Comprobar disponibilidad y capacidad. CU08 paga la reserva posteriormente; pagar no es un paso obligatorio para crearla.

## 6.7 CU07 — Consultar reservas y pagos propios

**Actores e iniciador:** Cliente

**Propósito:** Ver estados, fechas, importes y movimientos personales.

**Precondiciones:** Cliente activo autenticado.

**Postcondiciones:** Consulta de sus reservas y resúmenes financieros; sin modificación.

**Pantallas y salida:** /cliente/reservas y /cliente/historial

**Referencias:** RF04, RF05 · ClienteServlet.doGet/resumenes; ReservaServiceImpl.listarPropias; PagoServiceImpl.resumen/listarPropios

**Resumen:** El actor inicia consultar reservas y pagos propios; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Abre Mis reservas o Historial. | Consulta reservas del cliente y calcula sus resúmenes. |
| 2 | Selecciona pestaña, página o consulta movimientos. | Presenta estados, noches, total pactado, neto pagado y saldo; historial añade pagos propios. |

**Alternativas y errores:** 1a. Sin registros: estado vacío. Sesión vencida o rol incorrecto: filtros impiden acceso. No se incluyen reservas de otros clientes. La consulta no genera ni aprueba pagos.

**Relaciones:** Sin include/extend adicional.

## 6.8 CU08 — Pagar reserva

**Actores e iniciador:** Cliente

**Propósito:** Liquidar el saldo de una reserva pendiente mediante simulación local.

**Precondiciones:** Reserva propia PENDIENTE_PAGO no vencida; entrada futura; sin cruce ajeno; cliente activo autenticado.

**Postcondiciones:** Éxito: movimiento COBRO APROBADO y reserva CONFIRMADA en una transacción. Fallo: operación revertida; no se produce cargo parcial.

**Pantallas y salida:** /cliente/pagar?id=... → /cliente/reservas

**Referencias:** RF05 · ClienteServlet.doGet/doPost; PagoServiceImpl.resumen/registrar; PagoDaoJdbc.totalPagado/crear

**Resumen:** El actor inicia pagar reserva; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Abre pagar para su reserva. | Recupera reserva y muestra total pactado, pagado y saldo. |
| 2 | Elige cuenta registrada u otra solo para este pago, indica referencia opcional y confirma. | Bloquea reserva y valida pertenencia, estado, vencimiento, entrada, disponibilidad y cuenta bancaria. |
| 3 | Espera el resultado. | Recalcula saldo = total - neto aprobado, inserta COBRO con clave idempotente y lo aprueba en simulación. |
| 4 | Revisa su reserva. | Marca CONFIRMADA, confirma transacción y muestra aviso. |

**Alternativas y errores:** 2a. Reserva ajena, vencida, confirmada previamente o entrada alcanzada: rechazar. 3a. Referencia duplicada, banco inexistente, cuenta inválida o sobrecobro: rollback. La segunda petición no cobra de nuevo una reserva ya confirmada. TRANSFERENCIA, banco, número, tipo y uso REGISTRADA u OCASIONAL se conservan en cuenta_bancaria_pactada (JSONB). La cuenta ocasional no cambia las predeterminadas. Los movimientos antiguos sin método conservan ese dato como desconocido. No se contacta una pasarela.

**Relaciones:** Incluye Calcular saldo. CU07 permite consultar después los movimientos.

## 6.9 CU09 — Cancelar reserva propia

**Actores e iniciador:** Cliente

**Propósito:** Anular antes de la entrada una reserva personal y liberar fechas.

**Precondiciones:** Reserva propia pendiente o confirmada; entrada posterior a hoy; cliente activo autenticado.

**Postcondiciones:** Éxito: CANCELADA, ocupación liberada y devolución simulada de cobros aprobados pendientes de devolver. Historial conservado. Fallo: sin cambios parciales.

**Pantallas y salida:** /cliente/cancelar → /cliente/reservas

**Referencias:** RF04, RF05 · ClienteServlet.doPost; ReservaServiceImpl.cancelar; ServicioBase.cancelarPendiente

**Resumen:** El actor inicia cancelar reserva propia; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Solicita cancelar su reserva. | Valida sesión/CSRF, rol, pertenencia, estado y fecha de entrada. |
| 2 | Confirma la acción. | Actualiza a CANCELADA; BD libera ocupación. Recorre cobros aprobados y sus devoluciones previas. |
| 3 | Consulta el resultado. | Si hay importe por devolver registra DEVOLUCION aprobada ligada al cobro original; confirma todo atómicamente. |

**Alternativas y errores:** 1a. Reserva ajena, finalizada, expirada, cancelada o entrada alcanzada: rechazar. 3a. Reserva sin cobro aprobado: no crea devolución. Un error de persistencia revierte también la cancelación.

**Relaciones:** Registrar devolución simulada extiende Cancelar reserva propia si hay un cobro aprobado con saldo por devolver.

## 6.10 CU10 — Publicar inmueble

**Actores e iniciador:** Propietario

**Propósito:** Ofrecer un inmueble en alquiler por días.

**Precondiciones:** Propietario activo autenticado; ciudad del catálogo identificada; campos válidos; capacidad y tarifa positivas.

**Postcondiciones:** Éxito: nuevo inmueble PUBLICADO asociado al propietario, ciudad y moneda de su ubicación. Fallo: no se guarda publicación incompleta.

**Pantallas y salida:** /propietario/publicar → /propietario/inmuebles

**Referencias:** RF01 · PropietarioServlet.doPost; InmuebleServiceImpl.publicar; InmuebleDaoJdbc.crear

**Resumen:** El actor inicia publicar inmueble; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Abre publicar y completa datos. | Muestra catálogo de ciudades y formulario con título, dirección, descripción, capacidad y tarifa. |
| 2 | Selecciona tipo y entornos, o los deja pendientes, y envía. | Valida rol y datos, resuelve ciudad y asigna moneda desde ubicación. |
| 3 | Revisa su inventario. | Guarda inmueble PUBLICADO en transacción y muestra aviso. |

**Alternativas y errores:** 2a. Ciudad no reconocida o homónima sin identificar: solicitar selección. Tarifa/capacidad inválidas o clasificación desconocida: rechazar. NULL de tipo/entorno significa pendiente; no se infiere por título o ciudad. Las fotos son locales asignadas por id, sin subida de archivos.

**Relaciones:** Sin include/extend adicional.

## 6.11 CU11 — Consultar y editar inmuebles propios

**Actores e iniciador:** Propietario

**Propósito:** Mantener la información y clasificación del inventario propio.

**Precondiciones:** Propietario activo autenticado; para editar, inmueble perteneciente al actor.

**Postcondiciones:** Éxito: campos editables actualizados; se conservan estado y moneda. Reservas previas mantienen sus términos pactados. Fallo: cambios revertidos.

**Pantallas y salida:** /propietario/inmuebles y /propietario/editar?id=...

**Referencias:** RF01 · PropietarioServlet.doGet/doPost; InmuebleServiceImpl.listarPropios/editar

**Resumen:** El actor inicia consultar y editar inmuebles propios; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Abre sus inmuebles y elige editar. | Lista únicamente inventario propio y carga formulario. |
| 2 | Cambia campos y guarda. | Bloquea inmueble, verifica pertenencia y que capacidad no quede debajo de huéspedes de reservas vigentes. |
| 3 | Revisa los datos guardados. | Actualiza información y clasificación conservando estado/moneda; confirma transacción. |

**Alternativas y errores:** 1a. Inmueble ajeno: rechazar. 2a. Capacidad incompatible con reservas vigentes o campos inválidos: error y formulario conservado. Editar tarifa no cambia el precio histórico de reservas existentes.

**Relaciones:** Sin include/extend adicional.

## 6.12 CU12 — Desactivar inmueble propio

**Actores e iniciador:** Propietario

**Propósito:** Retirar una publicación del catálogo sin borrar contratos.

**Precondiciones:** Propietario activo autenticado; inmueble propio.

**Postcondiciones:** Inmueble INACTIVO; reservas y finanzas existentes se conservan.

**Pantallas y salida:** /propietario/desactivar → /propietario/inmuebles

**Referencias:** RF01 · PropietarioServlet.doPost; InmuebleServiceImpl.desactivar; InmuebleDaoJdbc.desactivar

**Resumen:** El actor inicia desactivar inmueble propio; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Solicita desactivar un inmueble propio. | Bloquea inmueble y valida propiedad. |
| 2 | Consulta su inventario. | Actualiza a INACTIVO y confirma; ya no aparece en búsqueda pública. |

**Alternativas y errores:** 1a. Inmueble ajeno/inexistente o actor inactivo: rechazar. Desactivar no cancela automáticamente las reservas previas. No se documenta una reactivación inexistente en las rutas actuales.

**Relaciones:** Sin include/extend adicional.

## 6.13 CU13 — Crear bloqueo

**Actores e iniciador:** Propietario

**Propósito:** Impedir alquiler en un intervalo de su inmueble.

**Precondiciones:** Propietario activo autenticado; inmueble propio; inicio no anterior a hoy; fin posterior; motivo no vacío de hasta 250 caracteres; sin cruces activos.

**Postcondiciones:** Éxito: ocupación BLOQUEO activa y subtipo bloqueo. Fallo: no se guarda intervalo parcial.

**Pantallas y salida:** /propietario/bloqueos → /propietario/bloquear

**Referencias:** RF08 · PropietarioServlet.doPost; BloqueoService.crear; BloqueoDaoJdbc.crear

**Resumen:** El actor inicia crear bloqueo; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Abre bloquear/liberar fechas y selecciona inmueble, intervalo y motivo. | Muestra inventario propio y bloqueos actuales. |
| 2 | Envía el bloqueo. | Comprueba perfil y datos; DAO valida propiedad y guarda ocupación más bloqueo. |
| 3 | Revisa el calendario. | BD impide cruces activos; confirma y presenta el bloqueo. |

**Alternativas y errores:** 2a. Inmueble ajeno, fechas inválidas o motivo vacío/largo: rechazar. 3a. Cruce con reserva o bloqueo activo: rollback. A diferencia de reservar, bloquear permite inicio en el día actual.

**Relaciones:** Sin include/extend adicional.

## 6.14 CU14 — Liberar bloqueo

**Actores e iniciador:** Propietario

**Propósito:** Volver a ofrecer fechas bloqueadas por el actor.

**Precondiciones:** Propietario activo autenticado; bloqueo asociado al propietario.

**Postcondiciones:** La ocupación del bloqueo queda inactiva; el registro se conserva.

**Pantallas y salida:** /propietario/liberar → /propietario/bloqueos

**Referencias:** RF08 · PropietarioServlet.doPost; BloqueoService.liberar; BloqueoDaoJdbc.liberar

**Resumen:** El actor inicia liberar bloqueo; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Elige liberar en un bloqueo de su listado. | Comprueba perfil y solicita actualización limitada al propietario. |
| 2 | Consulta el resultado. | Desactiva ocupación y muestra fechas liberadas. |

**Alternativas y errores:** 1a. Bloqueo inexistente, ajeno o error de persistencia: mensaje de error. La actualización DAO se limita al id y propietario; un id ajeno no libera fechas ajenas. No se reescriben fechas del bloqueo ni se borra historial.

**Relaciones:** Sin include/extend adicional.

## 6.15 CU15 — Consultar reservas recibidas y giros

**Actores e iniciador:** Propietario

**Propósito:** Consultar contratos y giros en los que figura como propietario pactado.

**Precondiciones:** Propietario activo autenticado.

**Postcondiciones:** Consulta de reservas recibidas y giros propios; sin modificar cobros ni giros.

**Pantallas y salida:** /propietario/reservas y /propietario/giros

**Referencias:** RF01, RF06 · PropietarioServlet.doGet; ReservaServiceImpl.listarRecibidas; GiroServiceImpl.listarPropios

**Resumen:** El actor inicia consultar reservas recibidas y giros; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Abre reservas recibidas. | Filtra reservas por propietarioPactado, no por un cambio posterior de dueño. |
| 2 | Abre giros. | Consulta giros asociados al beneficiario histórico y muestra montos/moneda/estado. |

**Alternativas y errores:** 1a/2a. Sin registros: estado vacío. El propietario no aprueba pagos, genera giros ni cancela reservas recibidas mediante estas pantallas; son consultas.

**Relaciones:** Sin include/extend adicional.

## 6.16 CU16 — Consultar gestión de rentas, reservas, pagos y giros

**Actores e iniciador:** Administrador

**Propósito:** Supervisar la operación y consultar estados e importes.

**Precondiciones:** Administrador activo autenticado.

**Postcondiciones:** Panel y listados consultados; no se alteran estados por abrir una página.

**Pantallas y salida:** /admin/panel, /admin/inmuebles, /admin/reservas, /admin/pagos, /admin/giros

**Referencias:** RF09 · AdministradorServlet.doGet; AdministracionServiceImpl.listarInmuebles/listarReservas/listarPagos/listarGiros

**Resumen:** El actor inicia consultar gestión de rentas, reservas, pagos y giros; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Abre el panel. | Comprueba rol y obtiene contadores y totales agrupados por moneda. |
| 2 | Abre un listado y aplica un estado. | Filtra inmuebles o reservas; en pagos/giros el filtro real usa el estado de la reserva asociada. |
| 3 | Revisa contratos y acciones disponibles. | Identifica reservas girables por fecha, estado, neto pagado y ausencia de giro. |

**Alternativas y errores:** 2a. Sin registros: estado vacío. No se suman COP y EUR como un importe único. La interfaz no ofrece editar manualmente movimientos aprobados; aprobación y devoluciones ocurren en el flujo de simulación.

**Relaciones:** Sin include/extend adicional.

## 6.17 CU17 — Cancelar reserva como administrador

**Actores e iniciador:** Administrador

**Propósito:** Anular un contrato elegible desde la gestión administrativa.

**Precondiciones:** Administrador activo autenticado; reserva pendiente o confirmada; entrada futura.

**Postcondiciones:** CANCELADA, fechas libres y devolución simulada si corresponde; historial conservado.

**Pantallas y salida:** /admin/cancelar → /admin/reservas

**Referencias:** RF09, RF05 · AdministradorServlet.doPost; AdministracionServiceImpl.cancelarReserva; ServicioBase.cancelarPendiente

**Resumen:** El actor inicia cancelar reserva como administrador; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Selecciona cancelar en una reserva. | Verifica administrador activo y bloquea reserva. |
| 2 | Espera la confirmación. | Valida estado/entrada, cancela, libera fechas y devuelve cobros aprobados pendientes de devolver en una transacción. |

**Alternativas y errores:** 1a. Rol incorrecto o reserva inexistente: rechazar. 2a. Entrada alcanzada o estado no cancelable: error. No exige que el administrador sea el cliente de la reserva.

**Relaciones:** Registrar devolución simulada extiende esta cancelación si existe importe aprobado por devolver.

## 6.18 CU18 — Desactivar inmueble como administrador

**Actores e iniciador:** Administrador

**Propósito:** Retirar un inmueble de la oferta desde la gestión de rentas.

**Precondiciones:** Administrador activo autenticado; inmueble existente.

**Postcondiciones:** INACTIVO; se conservan reservas y finanzas.

**Pantallas y salida:** /admin/desactivar → /admin/inmuebles

**Referencias:** RF09, RF01 · AdministradorServlet.doPost; AdministracionServiceImpl.desactivarInmueble

**Resumen:** El actor inicia desactivar inmueble como administrador; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Selecciona desactivar en el inventario administrativo. | Valida administrador y bloquea inmueble. |
| 2 | Revisa el listado. | Actualiza estado, confirma y presenta aviso. |

**Alternativas y errores:** 1a. Sin autorización o inmueble inexistente: rechazar. La acción no elimina inmueble ni cancela automáticamente contratos.

**Relaciones:** Sin include/extend adicional.

## 6.19 CU19 - Liquidar giro simulado al finalizar la estancia

**Actor principal:** Ciclo automatico del backend. Administrador supervisa y resuelve incidencias.

**Precondiciones:** Estancia terminada y renta FINALIZADA, pago aprobado completo, sin movimientos pendientes ni devoluciones aprobadas, tasa historica y cuenta valida del propietario pactado.

**Postcondiciones:** Giro EJECUTADO por el neto, con autor SISTEMA, fecha y cuenta historica. Si no se cumplen condiciones, registra PENDIENTE con motivo. Un fallo transaccional revierte el intento y registra FALLIDA; nunca marca como ejecutado un fallo.

**Referencias:** CicloReservas.ejecutar; CicloReservasDaoJdbc.ejecutar; liquidar_reserva_automatica; ComisionService; restricciones de giro y movimiento_pago.

| Paso | Sistema | Supervision |
|---|---|---|
| 1 | Al arrancar y cada minuto finaliza estancias y consulta rentas sin giro ejecutado. | No requiere abrir la aplicacion. |
| 2 | Bloquea cada renta y comprueba saldo, estado, tasa y cuenta del beneficiario pactado. | Consulta motivo de incidencias. |
| 3 | PENDIENTE → PROCESANDO → EJECUTADO; captura cuenta y registra resultado. | Consulta comision, neto, destino y autor. |
| 4 | Repite validaciones en cada reintento, sin duplicar giros. | Puede solicitar revision tras corregir una incidencia. |

La comision inicial es 30 %, editable desde Administracion > Giros para pagos nuevos. Se fija al primer cobro aprobado. Los giros humanos anteriores mantienen autorizacion y valores; el endpoint manual se conserva como compatibilidad, no es un requisito del flujo nuevo. No se transfieren fondos reales.

## 6.20 CU20 — Consultar historial anterior

**Actores e iniciador:** Administrador

**Propósito:** Acceder en solo lectura a contratos y finanzas del modelo previo.

**Precondiciones:** Administrador activo autenticado; tablas históricas conservadas en public.

**Postcondiciones:** Historial consultado; no se crea reserva, cobro ni giro en el modelo vigente.

**Pantallas y salida:** /admin/historial-anterior

**Referencias:** RF10 · AdministradorServlet.doGet; HistorialAnteriorService.consultar; HistorialAnteriorDaoJdbc.consultar

**Resumen:** El actor inicia consultar historial anterior; DayNest valida las condiciones y presenta el resultado descrito en el curso normal.

**Tipo:** principal, de usuario, real (interfaz existente).

| Paso | Acción del usuario | Respuesta del sistema |
|---|---|---|
| 1 | Abre Historial anterior. | Comprueba administrador y consulta tablas archivadas. |
| 2 | Revisa información previa. | Presenta registros históricos separados del esquema daynest. |

**Alternativas y errores:** 1a. Si no existe public.reserva.id_reserva, el DAO devuelve un listado vacío. Si existe ese contrato histórico pero faltan tablas relacionadas o falla la consulta, se muestra error de persistencia. No se migra ni recrea información al abrir la pantalla. El historial no debe interpretarse como contratos nuevos sin comisión.

**Relaciones:** Sin include/extend adicional.

## 6.21 Comportamientos reutilizados incluidos y extendidos

| Identificador auxiliar | Iniciador y tipo | Condición y resultado | Referencia real |
|---|---|---|---|
| AUX01 Comprobar disponibilidad y capacidad | CU06 lo incluye; secundario, de sistema | Comprobar huéspedes, publicación, fechas y ocupaciones; rechazar antes de guardar si no cumplen | ReservaServiceImpl.crear; ReservaDaoJdbc.existeCruce; restricciones GiST/capacidad |
| AUX02 Calcular saldo | CU08 lo incluye; secundario, de sistema | Total pactado menos neto de cobros/devoluciones aprobados; mostrar y recalcular al pagar | PagoServiceImpl.resumen/registrar; PagoDaoJdbc.totalPagado |
| AUX03 Registrar devolución simulada | Extiende CU09 y CU17; secundario, de sistema | Punto de extensión: tras cancelar, dentro de la misma transacción; solo si hay cobro aprobado con importe por devolver | ServicioBase.cancelarPendiente; PagoDaoJdbc.listarPorReserva/crear |

Los comportamientos auxiliares no son tres pantallas adicionales ni métodos inventados con esos nombres. Se expresan como resultados funcionales y se realizan en las operaciones Java indicadas. En AUX03 la devolución es obligatoria cuando se cumple la condición; no es una decisión del cliente ni una llamada a un banco.

# 7. Verificación y demostración

El comprobador local compara nombres de clases, interfaces, atributos y métodos, además de las 19 tablas y 120 columnas con PK/FK/UQ. PlantUML valida sintaxis y genera las imágenes localmente. Este control no sustituye una revisión semántica: las relaciones, condiciones, rutas y secuencias se contrastaron con servlets, servicios, DAO y sql/instalacion/schema.sql durante la actualización.

Las pruebas funcionales y de persistencia existentes cubren fechas y capacidad, bloqueo y concurrencia, intervalos adyacentes, términos históricos, pagos, devolución, idempotencia, expiración y giro neto. El registro previo de la implementación informa 65 pruebas satisfactorias; corresponde a aquella ejecución, no a una nueva ejecución causada por este informe. La presente actualización es documental y no modifica código Java ni datos operativos.

Para la demostración se recomienda seguir docs/GUION_DEMO.md: iniciar PostgreSQL y Jetty, buscar una ciudad, mostrar la distinción entre Cartagena Colombia/España, publicar o editar como propietario, reservar como cliente, pagar, consultar el registro en BD y mostrar la gestión administrativa. El giro requiere que la salida ya se haya cumplido; las pruebas automatizadas usan una estancia histórica en esquema temporal, sin fabricar una reserva pasada en la base activa. No reiniciar ni precargar de nuevo la base para ejecutar la demostración.

| Comprobación manual | Resultado esperado |
|---|---|
| Buscar Bogota sin tilde o elegir Bogotá del catálogo | Coincidencias de la ciudad seleccionada |
| Elegir Cartagena con región y país | Colombia y España se distinguen por id |
| Reservar por encima de capacidad o con cruce activo | Error y ausencia de reserva parcial |
| Reservar intervalo adyacente | Permitido si las demás condiciones se cumplen |
| Pagar una reserva propia válida | Cobro aprobado simulado y reserva confirmada |
| Cancelar antes de entrada una reserva pagada | Cancelada, fechas libres y devolución registrada |
| Intentar repetir pago/giro o actuar sobre reserva ajena | Rechazo; sin duplicar operaciones válidas |
| Consultar desde PC y teléfono conectados al mismo servidor | Datos compartidos en PostgreSQL; actualizar vista para ver cambios |

# 8. Entregables y preparación final

Se incluyen fuentes Java/JSP/CSS/JS y SQL del proyecto, fuentes PlantUML y SVG locales, fichas ampliadas y este informe en Markdown, HTML y PDF. El HTML y el PDF incorporan imágenes del diseño; el Markdown permanece editable. El archivo WAR se genera mediante Maven según README.md.

Antes de entregar, completar nombres del grupo, asignatura, institución y docente en la portada; revisar si el profesor solicita una plantilla o campos adicionales; y realizar una demostración en el computador que se llevará. Clasificar los inmuebles pendientes permitirá que las categorías cabaña/campo/mar tengan datos confirmados; una categoría vacía no demuestra una falla del filtro.

# 9. Referencias

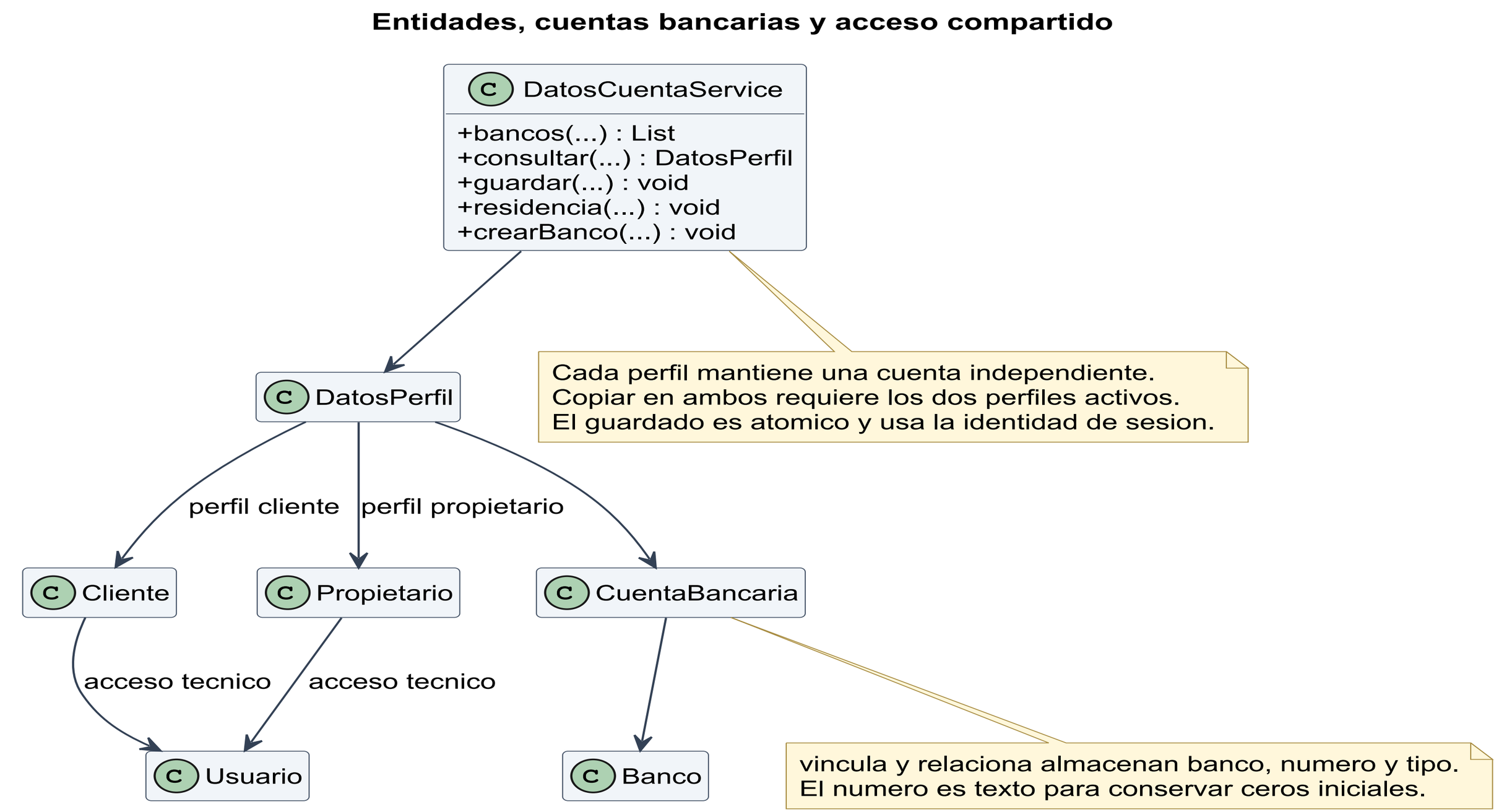

- Enunciado proporcionado por el usuario: Proyecto – Renta de inmuebles por días, entrega del 21 de noviembre de 2026.
- Álvaro E. Ortiz Dávila, material docente «3 - Diagrama de clases.pdf», 22 páginas: notación de atributos, operaciones, interfaces, relaciones y multiplicidades. Archivo de consulta externo al proyecto.
- Álvaro E. Ortiz Dávila, material docente «4 - Casos de Uso.pdf», 31 páginas: actores, relaciones include/extend y fichas ampliadas, especialmente páginas 12–13 y 27. Archivo de consulta externo al proyecto.
- Código fuente local: src/main/java/co/edu/rentainmuebles (model, servlet, service, dao, filter, config); src/main/webapp/WEB-INF/views; schema.sql.
- Documentación del proyecto: MODELO_DEFINITIVO.md, ADAPTACION_SVG.md, GUION_DEMO.md y diseno/CREDITOS_IMAGENES.md.
- Herramienta de diagramación: PlantUML 1.2025.10, JAR obtenido de Maven Central; renderizado local con Smetana y Graphviz integrado en Windows para casos de uso. Su ubicación y checksum se documentan en diagramas/README.md.

# Datos bancarios y residencia

Desde Mi cuenta → Mis datos y cuentas, el cliente y el propietario guardan banco, número y tipo de cuenta en sus relaciones independientes. La casilla de copia guarda los mismos datos en ambos perfiles de forma atómica; no activa perfiles ni modifica cuentas de otras personas. La residencia del cliente se selecciona por ciudad. La administración gestiona bancos y países del catálogo. Los datos desconocidos de registros migrados aparecen pendientes, sin inventar cuentas ni ubicaciones.

### Cuentas y entidades independientes

El precio por noche pertenece a ASOCIA. El inmueble incluye área en m² y habitaciones, editables por su propietario, sin alterar las tarifas pactadas de rentas previas. Los cobros nuevos conservan la cuenta bancaria registrada u ocasional utilizada. Los giros conservan la cuenta del propietario al ejecutarse; posteriores cambios de perfil no reescriben ese historial. La cuenta ocasional no reemplaza la predeterminada y su eventual devolución conserva la cuenta del cobro original. Los pagos siguen simulados.



# Extension financiera y estado de activacion

El informe tecnico GIROS_AUTOMATICOS.md detalla configuracion versionada, tasa historica, origen SISTEMA, cuenta al ejecutar, auditoria, pruebas y plan de migracion. La nomenclatura esta aplicada. La extension financiera se probo en copias aisladas y se aplico con respaldo el 17 de octubre de 2026; conserva 80 inmuebles y el historial original.

# Comisiones e ingresos por inmueble

Las nuevas consultas financieras usan `IngresosServlet` e `IngresosService`, sin tablas ni columnas adicionales. Se agregan los pagos por renta antes de unirlos con un giro vigente; se evitan duplicados por múltiples cobros, devoluciones o intentos fallidos. Las comisiones y netos liquidados provienen de giros EJECUTADOS, usando sus importes históricos. Los pendientes son estimaciones sobre reservas pagadas por completo, confirmadas o finalizadas, con tasa fijada, sin devoluciones aprobadas ni movimientos pendientes y aún no liquidadas.

Cada inmueble aparece una vez por moneda, incluidos los que no tienen ingresos. Una consulta por propietario conserva el beneficiario pactado en la renta, aunque cambie el dueño actual del inmueble. Los resultados se filtran en SQL por usuario autorizado e inmueble. Los totales y el detalle se leen en una transacción de solo lectura con una misma instantánea; el historial tiene páginas de 20 filas y los acumulados abarcan todo el filtro.

## CU21 — Consultar comisiones por inmueble y arriendo

**Actor:** Administrador activo. **Propósito:** conocer las comisiones de DayNest liquidadas y las pendientes estimadas. **Precondición:** sesión y permiso administrativo. **Poscondición:** consulta sin alterar operaciones. **Pantalla:** `/admin/ingresos`, accesible desde Comisiones e ingresos, panel y Giros.

| Acción del administrador | Respuesta del sistema |
|---|---|
| Abre el apartado. | Presenta acumulados por moneda y resumen por inmueble. |
| Selecciona un inmueble. | Filtra los importes y el historial por ese inmueble. |
| Consulta cada arriendo o cambia de página. | Muestra cliente, propietario, fechas, estado, pago neto, tasa, comisión, neto al propietario y situación del giro; conserva los acumulados completos del filtro. |

**Alternativas:** lista vacía informada; identificador inválido o página no numérica rechazados; las páginas fuera de rango se ajustan; usuario sin permiso rechazado. Una comisión estimada no se muestra como ingreso liquidado. COP y EUR no se suman entre sí. No representa un saldo bancario o billetera.

## CU22 — Consultar ingresos propios por inmueble y arriendo

**Actor:** Propietario activo. **Propósito:** conocer cuánto se liquidó a su favor y cuánto está pendiente por cada propiedad. **Precondición:** sesión y perfil propietario. **Poscondición:** consulta restringida al propietario, sin alterar operaciones. **Pantalla:** `/propietario/ingresos`, accesible desde Mis ingresos y Mis giros.

| Acción del propietario | Respuesta del sistema |
|---|---|
| Abre Mis ingresos. | Presenta netos liquidados y pendientes estimados, separados por moneda, y comisiones correspondientes. |
| Selecciona una propiedad. | Muestra sus arriendos y el neto individual de cada uno, sin exponer operaciones de otro propietario. |

| Acción del propietario | Respuesta del sistema |
| --- | --- |
| Consulta el historial o vuelve a todos los inmuebles. | Conserva precios y tasas históricos; incluye inmuebles sin rentas y operaciones donde fue beneficiario pactado. |

**Alternativas:** filtro sobre un inmueble ajeno no devuelve sus ingresos; perfil inactivo o no autorizado rechazado. Reservas canceladas y devueltas no generan ganancias; pagos incompletos no se presentan como ganancias pendientes. Los giros fallidos no suman ingresos liquidados.